TABLE OF CONTENTS

1. Introduction and Objectives....3
1.1 Title of the Project....3
1.2 Introduction....3
1.3 Evolution of the Project since the Synopsis....4
1.4 Objectives of the Project....5
1.5 Project Category....6
1.6 Tools, Platform and Environment Used....6
1.7 Hardware Requirements....7
1.8 Software Requirements....7
2. System Analysis....9
2.1 Identification of Need....9
2.1.1 Problem Definition....9
2.1.2 The Existing System....9
2.1.3 The Proposed System....10
2.1.4 Feasibility Study....11
2.2 Project Planning and Project Scheduling....12
2.2.1 Project Plan....12
2.2.2 Activity Estimates and Critical Path (PERT Analysis)....13
2.2.3 PERT Chart....14
2.2.4 Gantt Chart....16
2.3 Software Requirement Specification (SRS)....18
1. Introduction....18
2. General Description....19
3. Specific Requirements....22
2.4 Software Engineering Paradigm Applied....25
2.5 Data Models and Analysis Diagrams....27
2.5.1 Data Flow Diagrams (DFD)....27
2.5.2 Control Flow Diagram....33
2.5.3 State Diagrams....34
2.5.4 Sequence Diagrams....39
2.5.5 Entity Relationship Model....41
2.5.6 Class Diagram....43
2.5.7 Use Case Diagram....45
2.5.8 Activity Diagrams....47
3. System Design....49
3.1 Architecture Overview....49
3.2 Modularization Details....51
Module descriptions....51
Source code organisation....54
3.3 Data Integrity and Constraints....55
3.4 Database Design....56
3.5 Procedural Design (Process Logic)....64
Object-Oriented Design....66
3.6 User Interface Design....67
3.7 Test Case Design....68
4. Standardization of the Coding....70
4.1 Naming and Layout Conventions....70
4.2 Code Efficiency....70

4.3 Error Handling....71
4.4 Parameters Calling and Passing....72
4.5 Validation Checks....73
5. Testing....74
5.1 Testing Techniques and Testing Strategies Used....74
5.1.1 Testing Techniques....74
5.1.2 Testing Strategies....75
5.2 Testing Plan Used....76
5.3 Test Reports....76
5.3.1 Unit Test Cases....76
5.3.2 System Test Cases....80
5.4 Debugging and Code Improvement....82
6. System Security Measures....84
6.1 Database and Data Security....84
6.2 Creation of User Profiles and Access Rights....86
7. Reports....88
7.1 Project Report (CSV Download)....88
7.2 Project Progress Document and Report Document....89
7.3 Task Progress Document....90
7.4 Gantt Chart....90
7.5 Dashboard Statistics....91
7.6 Notification E-mail....91
8. Future Scope and Further Enhancement of the Project....92
8.1 Limitations of the Present System....92
8.2 Future Enhancements....92
9. Bibliography....94
Appendices....96
Appendix A — Input and Output Screens....96
Appendix B — API Endpoint Reference....97
Appendix C — Installation and Running....98
Appendix D — Scheduled Jobs....100
Glossary....101

1. INTRODUCTION AND OBJECTIVES

1.1 Title of the Project

TRACE — Task and Resource Attendance and Comprehensive Evaluation System

An AI-driven project and task management platform with GitHub integration.

1.2 Introduction

Every software project begins as a pile of requirement documents — a client brief, a feature list, meeting notes, a scope statement — and ends, if it succeeds, as a set of finished pieces of work that someone can point to. Between those two ends lies the part that usually goes wrong: turning the documents into a plan, splitting the plan into tasks, deciding who does what, keeping a schedule honest while work slips, and telling management how far along things really are. In most organisations this is done by hand, with spreadsheets, a chat group and a Gantt chart that stops being true the week after it is drawn.

TRACE is a web-based platform that automates that middle part. A Manager creates a project and uploads its raw requirement documents (PDF, Word, Markdown or plain text). With one action — **Evaluate Project** — an AI pipeline reads the documents, writes a structured project document, splits the work into task groups (modules) with duration estimates and dependencies, and draws the first version of a Gantt chart. From there the project is handed to people: the Manager appoints a Project Manager (with AI-ranked suggestions), the Project Manager creates tasks (each of which automatically becomes an issue in the project's own GitHub repository), assigns them to Members (again with AI-ranked suggestions), and the Members do the work and mark their tasks complete.

The system then keeps itself up to date. Every completion, every missed deadline and every change to the project document re-plans the Gantt chart as a new, auditable version. Every week the AI reads each task's progress document and rewrites the project's progress report for the Manager. Notifications reach the right people in real time over a WebSocket and by e-mail, and each task has its own discussion thread between the person doing it and the person who assigned it.

TRACE also keeps a complete **workforce record set** — employees, departments, designations, leaves, holidays, attendance and work shifts — owned and maintained by an Administrator, because planning and assignment need to know who exists and who is

available. The Administrator additionally has a raw control panel over every table, a live service monitor and a configuration screen for GitHub and mail settings.

The platform is built with a **React** single-page frontend, a **FastAPI** (Python) backend on **PostgreSQL**, **Celery** workers on **Redis** for background and AI jobs, **MinIO** object storage for uploaded files, **GitHub** for issues and generated documents, and an **OpenAI-compatible** large language model for the AI steps.

1.3 Evolution of the Project since the Synopsis

The approved synopsis described TRACE as an AI-powered *attendance* and productivity system centred on face recognition, GPS geo-fencing, IP tracking and a weighted productivity score. During analysis and the first build iterations it became clear that the part of the problem worth automating was not attendance but **project delivery**: attendance data is easy to record and hard to act on, while the path from requirement documents to a tracked, scheduled set of tasks is where teams lose the most time. The scope was therefore re-specified as follows, keeping the name and the underlying technology stack of the synopsis:

Table 1: Synopsis scope versus delivered scope

Area	In the synopsis	As delivered
Core feature	Attendance with face recognition and geo-fencing	AI-driven project evaluation, task planning and Gantt scheduling
Attendance	Employee clock-in / clock-out with GPS validation	Retained as Admin-maintained records (bulk upload); no self-service capture
Tasks	Task CRUD with in-browser timer	Project → Task Group → Task hierarchy; every task is a GitHub issue
AI	Weighted productivity score, anomaly flags	LLM pipeline: project document, task groups, Gantt, people ranking, task plans, weekly progress reports
Productivity score	Attendance + tasks + active time	Re-based on task signals only (throughput and on-time rate); used as an input to AI ranking
Reporting	Weekly e-mailed PDF report	AI-written progress and report documents; CSV report download for a date range
Real time	WebSocket live attendance	WebSocket notifications and task discussions
Roles	Admin, Manager, Employee	Same three platform roles, plus per-project roles (Project Manager, Member)

The technology choices of the synopsis — React, FastAPI, PostgreSQL, SQLAlchemy with Alembic, JWT, Celery with Redis, WebSockets, SMTP mail, Git/GitHub and Docker Compose — were all kept.

1.4 Objectives of the Project

The objectives of TRACE are:

- **Automate project planning from raw documents.** Convert uploaded requirement files into a structured project document, a set of task groups with realistic durations and dependencies, and a first Gantt chart — without manual data entry.
- **Keep the schedule honest automatically.** Regenerate the Gantt chart as a new version whenever a task is completed, becomes overdue or the project document changes, recording the reason for every change.
- **Make every task a real work item where the code lives.** Create one GitHub repository per project and one GitHub issue per task, and keep the task and the issue in step (assignment, completion, cancellation, deletion).
- **Assist people decisions without taking them.** Rank candidate Project Managers and task assignees with AI, using live performance signals, but always leave the decision to a human.
- **Put completion in the hands of the person doing the work.** Let the assignee mark their own task complete, let the Project Manager set any status, and enforce this rule on the server.
- **Report progress without meetings.** Maintain per-task progress documents and a weekly AI-written project progress report, and allow a CSV report of the project tree to be downloaded for any date range.
- **Deliver the right information to the right person in real time**, through in-app notifications, e-mail and per-task discussion threads.
- **Separate the three audiences cleanly** — Administrator, Manager, Employee — each with its own panel, with authorisation enforced at two layers on the server.
- **Retain a complete, Admin-owned workforce record set** with CRUD and bulk upload, as the basis for planning and availability.
- **Be secure, testable and maintainable**: hashed passwords, JWT authentication, encrypted secrets, schema-validated AI output, database migrations as the only schema owner, and an automated test suite over every route.

1.5 Project Category

TRACE belongs to the following categories:

- **Web Technology / Cloud Computing (SaaS)** — a browser-based single-page application backed by a REST API and WebSockets, deployable to a cloud server and published through a secure tunnel.
- **Artificial Intelligence** — it uses a large language model (LLM) for requirement analysis, planning, ranking and report writing, with strict schema validation of every AI response.
- **RDBMS** — all business data lives in a normalised PostgreSQL database with primary keys, foreign keys, unique and not-null constraints, managed through versioned migrations.
- **Object-Oriented Programming** — the backend is built from Python classes (ORM models, service classes, access-control objects, Pydantic schemas) and the frontend from reusable React components.

1.6 Tools, Platform and Environment Used

Table 2: Software stack

Component	Technology	Purpose
Frontend	React 19 (JSX) with Vite	Single-page user interface for the three panels
UI library	shadcn/ui, Tailwind CSS v4, lucide-react icons	Consistent components, light and dark themes
Graph / chart	@xyflow/react (tree graph), hand-built SVG Gantt	Project tree and Gantt visualisation
Backend framework	FastAPI (Python 3)	REST API, validation, dependency injection
ASGI server	Uvicorn	Serves the FastAPI application
Database	PostgreSQL	Relational data storage
ORM and migrations	SQLAlchemy 2.0 (async) + Alembic	Models, queries and versioned schema changes
Validation	Pydantic V2	Request, response and AI-output schemas
Authentication	JWT (PyJWT) + bcrypt	Stateless tokens and password hashing
Secret encryption	Fernet (cryptography)	Encrypts stored tokens and passwords at rest
Background jobs	Celery 5 (worker + beat) on Redis	AI jobs, scheduled sweeps, e-mail
Real time	WebSocket + Redis publish/subscribe	Live notification and discussion events
Object storage	MinIO (S3 compatible)	Uploaded project source files

Component	Technology	Purpose
Document storage	GitHub repository files	Generated Markdown documents with history
Issue tracking	GitHub REST API	One issue per task
AI	OpenAI SDK (any OpenAI-compatible model)	Evaluation, planning, ranking, reports
Text extraction	pypdf, python-docx	Reading uploaded PDF and Word files
Mail	SMTP (Mailpit in development)	Notification e-mails
Containers	Docker Compose	PostgreSQL, Redis, MinIO, Mailpit, pgAdmin
Testing	pytest, pytest-asyncio	Automated backend tests
Version control	Git + GitHub	Source control
Editor / tools	VS Code, pgAdmin, browser developer tools	Development and inspection

1.7 Hardware Requirements

Table 3: Hardware requirements

Requirement	Specification
Developer machine	Intel Core i5 (8th generation) or equivalent, 8 GB RAM (16 GB recommended), 20 GB free disk space
Server (deployment)	2 vCPU, 4 GB RAM, 40 GB SSD (for example a cloud virtual machine)
Client machine	Any computer able to run a current web browser; 4 GB RAM
Display	1366 × 768 or higher (the interface is designed for desktop screens)
Network	Broadband Internet connection (needed for GitHub and the AI model)

1.8 Software Requirements

Table 4: Software requirements

Requirement	Specification
Operating system (server)	Linux (Ubuntu 22.04 or later) or any OS that runs Docker
Operating system (client)	Platform independent — Windows, Linux or macOS
Runtime	Python 3.11 or later, Node.js 20 or later
Containers	Docker Engine with Docker Compose
Browser	Google Chrome, Mozilla Firefox, Microsoft Edge (current versions)

Requirement	Specification
External accounts	A GitHub Organization with a fine-grained access token; an OpenAI-compatible API key

2. SYSTEM ANALYSIS

System analysis is the study of the problem the software must solve, the environment it will work in and the requirements it must meet, before any design decision is taken. This chapter identifies the need for TRACE, lays out the project plan and schedule, specifies the requirements in the IEEE 830 format, names the software engineering paradigm followed, and presents the analysis models of the system.

2.1 Identification of Need

2.1.1 Problem Definition

A study of how small and medium software teams plan and track their projects showed the following recurring problems:

- **Requirements are read, not processed.** Requirement documents arrive as PDFs, Word files and notes. Someone has to read all of them and re-type the scope into a plan. The effort is large, and the result depends on who did it.
- **Plans go stale immediately.** A Gantt chart drawn at kick-off is not updated when tasks finish early, slip or are added. Within weeks it describes a project that no longer exists, and nobody can say *why* a date moved.
- **Task trackers and code live apart.** Tasks are kept in a spreadsheet or a separate tool while the code and its issues live on GitHub. The two drift apart: a task closed in one place stays open in the other.
- **Assignment is guesswork.** Choosing a Project Manager or an assignee depends on memory and impressions rather than on the person's actual delivery record.
- **Completion is unclear.** It is often not recorded who declared a task done, and when, and the rules for who may do so are enforced only socially.
- **Progress reporting is manual.** Managers receive progress by asking in meetings or reading long chat histories; weekly status reports take hours to compile.
- **Communication is scattered.** Discussion about one task is buried in a project-wide chat, and people learn about assignments or deadlines late.
- **Workforce data is disconnected from planning.** Employee records, leaves, holidays and shifts are kept in HR spreadsheets that the planning process never sees.

2.1.2 The Existing System

In the organisations studied, project delivery was managed with a combination of general-purpose tools:

- requirement documents in shared folders;
- a spreadsheet or a generic board tool for the task list;
- a manually drawn Gantt chart in a spreadsheet or presentation;
- GitHub (or a similar service) for the code and, sometimes, issues;
- a group chat for all communication;
- HR records in separate spreadsheets.

Limitations of the existing system

- No single source of truth: the same task exists in two or three places.
- High manual effort to create and maintain the plan; no automation at all.
- No history of schedule changes and no reason recorded for them.
- No enforcement of who may change a task's status.
- No objective basis for choosing people.
- Reports must be written by hand and are out of date when read.
- Security depends on each tool separately; there is no role-based view of the project.

2.1.3 The Proposed System

TRACE replaces the collection of tools with one platform built around the project:

- The Manager uploads the raw requirement documents; **the AI turns them into a project document, task groups and a Gantt chart**.
- Every task **is** a GitHub issue in a repository TRACE creates for the project, so the task list and the code's issue list cannot drift apart.
- The Gantt chart is **regenerated as a new version** whenever something changes, with the reason recorded.
- **AI ranks candidates** for Project Manager and for each task from live delivery data; a human decides.
- **Status authority is enforced by the server**: the assignee sets their own task's status, including *Completed*; the Project Manager may set any status; nobody else can.
- **The AI writes the weekly progress report**, and any user can download a CSV report of the project tree for a date range.
- Notifications and per-task discussions keep communication attached to the work it is about.
- The Administrator maintains the workforce record set in the same system, with bulk upload.

Advantages of the proposed system

- Large reduction of manual planning effort; a plan exists minutes after the documents are uploaded.
- One source of truth for tasks, shared with the code repository.
- An auditable schedule history.
- Clear, enforced accountability for task completion.
- Fair, data-based suggestions for people decisions.
- Up-to-date reporting with no meetings needed.
- Clean separation of Administrator, Manager and Employee concerns.

2.1.4 Feasibility Study

A feasibility study checks whether the proposed system can be built and used with the available resources. TRACE was examined from five angles.

Technical feasibility

All components of TRACE are mature, well-documented open-source technologies: React, FastAPI, PostgreSQL, Redis, Celery, MinIO and Docker. GitHub provides a stable REST API for repositories, issues, collaborators and file commits. Large language models are available through a standard, OpenAI-compatible API that returns JSON on request. The main technical risk — that an AI model returns malformed or invalid output — is controlled by validating every response against a declared schema and retrying with the validation errors. The system is therefore **technically feasible**.

Economic feasibility

All software used is free and open source. GitHub Organizations with public repositories are free. The only running costs are a small cloud server (or an existing machine) and the AI model's usage charges, which are incurred only when an AI action runs (evaluation, suggestions, task plans, weekly reports) and not on every page view. Compared with the hours of manual planning and reporting it saves, the system is **economically feasible**.

Operational feasibility

Users work in a web browser and need no installation. Each role sees only its own panel with a small number of clear actions: a Manager creates a project and presses *Evaluate Project*; a Project Manager creates and assigns tasks; a Member updates the status of their own tasks. Developers keep working on GitHub, where their tasks already appear as issues. The system fits the way teams already work and is **operationally feasible**.

Schedule feasibility

The work was divided into eleven activities with a critical path of 84 days (Section 2.2), within the time available for the MCA project. The phased build allowed each part to be finished and tested before the next depended on it. The project is **feasible within schedule**.

Legal and ethical feasibility

All libraries are used under their open-source licences. Repositories created by TRACE are public by design (an educational choice), and the interface states this clearly where requirement documents are uploaded; uploaded source files themselves stay private in MinIO. The AI only *suggests* people and never assigns them, so accountability for people decisions stays with a human. Attendance data is recorded but deliberately not used to rank people.

2.2 Project Planning and Project Scheduling

2.2.1 Project Plan

Project planning divides the work into activities, estimates each one, and orders them by their dependencies. TRACE was planned as a sequence of build phases, each ending in something that can be demonstrated and tested, grouped into the activities below.

Table 5: Work breakdown of the project

Id	Activity	Work included
A	Requirement analysis and feasibility study	Study of the problem, existing tools, user roles; feasibility study
B	Synopsis and SRS	Project synopsis, Software Requirement Specification (IEEE 830)
C	System design	Database schema, architecture, API surface, UI wireframes, AI pipeline design
D	Foundation (build phases 0–2)	Clean-up, schema and migrations, JWT authentication, three roles, three panels, route guards, landing page
E	Projects, tasks and GitHub (phases 3, 3a)	Projects, source files, text extraction, task groups, tasks as GitHub issues, status authority, repository creation, collaborator invitations, encrypted settings
F	Visualisation (phase 4)	Project tree graph, Gantt chart data schema, Gantt renderer, version history
G	AI pipeline (phase 5)	AI service layer, Evaluate Project, suggestions, task progress documents, Gantt re-planning, weekly progress report, re-based scoring
H	Notifications and discussion (phase 7)	Notification centre, e-mail, WebSocket live events, per-task discussions, due and overdue reminders

Id	Activity	Work included
I	Administration (phase 8)	Mail configuration, Workforce with bulk upload, Control Panel, Monitor, report download
J	Testing and debugging	Unit, integration and system testing; role matrix; migration tests; fixes
K	Documentation and deployment	Project report, README, production configuration, secure tunnel deployment

2.2.2 Activity Estimates and Critical Path (PERT Analysis)

The Programme Evaluation and Review Technique (PERT) models the project as a network of activities. For each activity the **earliest start (ES)**, **earliest finish (EF)**, **latest start (LS)** and **latest finish (LF)** are computed by a forward and a backward pass; the difference LS − ES is the **slack**. Activities with zero slack form the **critical path** — any delay on them delays the whole project.

In the table, *t* is the expected duration and *Pred.* the predecessor activities. The durations are expected times $t = (o + 4m + p) / 6$ computed from optimistic (o), most likely (m) and pessimistic (p) estimates, rounded to whole days.

Table 6: Activity estimates, predecessors and PERT times (days)

Id	Activity	o	m	p	t	Pred.	ES	EF	LS	LF	Slack
A	Requirement analysis and feasibility	5	7	9	7	—	0	7	0	7	0
B	Synopsis and SRS	5	7	9	7	A	7	14	7	14	0
C	System design	7	10	13	10	B	14	24	14	24	0
D	Foundation (phases 0–2)	7	10	13	10	C	24	34	24	34	0
E	Projects, tasks and GitHub (3, 3a)	10	14	18	14	D	34	48	34	48	0
F	Visualisation (phase 4)	5	7	9	7	E	48	55	55	62	7
G	AI pipeline (phase 5)	10	14	18	14	E	48	62	48	62	0
H	Notifications and discussion (phase 7)	5	7	9	7	F, G	62	69	62	69	0
I	Administration (phase 8)	7	10	13	10	D	34	44	59	69	25
J	Testing and debugging	7	10	13	10	H, I	69	79	69	79	0
K	Documentation and deployment	3	5	7	5	J	79	84	79	84	0

Critical path: A → B → C → D → E → G → H → J → K, with a total expected duration of **84 days**. Visualisation (F) has 7 days of slack because it runs in parallel with

the longer AI pipeline (G); Administration (I) has 25 days of slack because it depends only on the foundation and can proceed alongside the core work.

2.2.3 PERT Chart

In the PERT chart each node is an event, numbered in order, and each arrow is an activity labelled with its id and expected duration. Dashed arrows are dummy activities that carry a dependency but take no time. The critical path is drawn in bold.

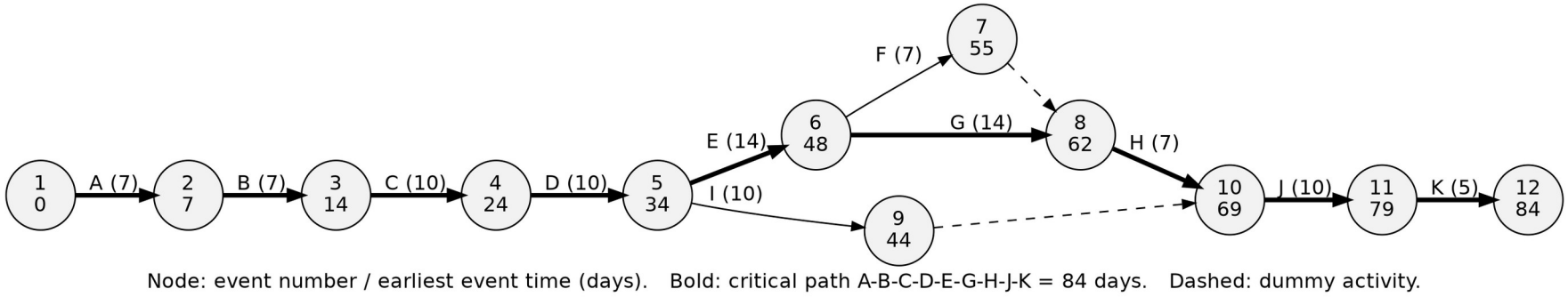


Figure 1: PERT chart of the TRACE project (critical path in bold)

2.2.4 Gantt Chart

The Gantt chart places each activity on a calendar. The project was scheduled to start on **5 July 2026** and, following the critical path, to finish on **26 September 2026**.

Table 7: Project schedule (calendar dates)

Id	Activity	Start	Finish	Duration
A	Requirement analysis and feasibility	05 Jul 2026	11 Jul 2026	7 days
B	Synopsis and SRS	12 Jul 2026	18 Jul 2026	7 days
C	System design	19 Jul 2026	28 Jul 2026	10 days
D	Foundation (phases 0–2)	29 Jul 2026	07 Aug 2026	10 days
E	Projects, tasks and GitHub (3, 3a)	08 Aug 2026	21 Aug 2026	14 days
F	Visualisation (phase 4)	22 Aug 2026	28 Aug 2026	7 days
G	AI pipeline (phase 5)	22 Aug 2026	04 Sep 2026	14 days
H	Notifications and discussion (phase 7)	05 Sep 2026	11 Sep 2026	7 days
I	Administration (phase 8)	08 Aug 2026	17 Aug 2026	10 days
J	Testing and debugging	12 Sep 2026	21 Sep 2026	10 days
K	Documentation and deployment	22 Sep 2026	26 Sep 2026	5 days
	Total (critical path)	**05 Jul 2026**	**26 Sep 2026**	**84 days**

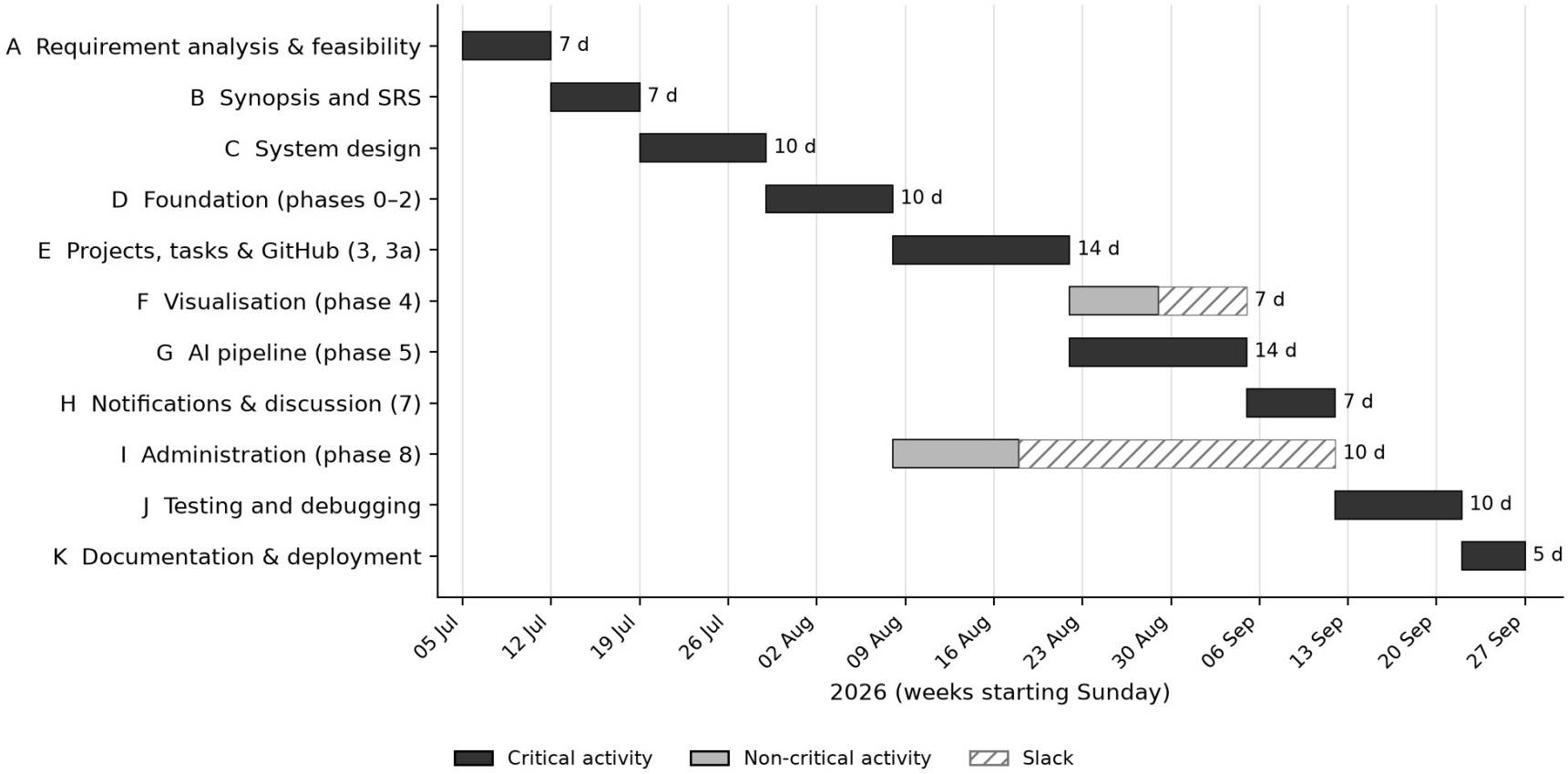


Figure 2: Gantt chart of the TRACE project

2.3 Software Requirement Specification (SRS)

The Software Requirement Specification below follows the structure of **IEEE Standard 830-1998**, *IEEE Recommended Practice for Software Requirements Specifications*.

1. Introduction

1.1 Purpose of the Software Requirement Specification

This document describes the features, behaviour and constraints of TRACE. It serves as a guide for the developers on one hand and as a validation reference for the users and evaluators on the other. It covers what the system must do for each of its three kinds of user — Administrator, Manager and Employee — how the AI pipeline behaves, and the quality attributes the system must have.

1.2 Scope of the Product

TRACE is a web application that takes a software project from its raw requirement documents to a delivered, tracked set of GitHub issues, with automatically maintained documentation, a self-updating Gantt chart and progress reports. In scope:

- projects, their uploaded source documents and their AI-generated living documentation;
- task groups (Gantt modules) and tasks, where each task is a GitHub issue;
- AI automation: project evaluation, task suggestion, assignee and Project Manager suggestion, Gantt generation and re-planning, task progress documents and weekly progress reports;
- manual task completion by the assignee, with server-enforced status authority;
- per-task discussion, notifications, progress and completion reporting;
- an Admin-owned workforce record set (employees, departments, designations, leaves, holidays, attendance, work shifts).

Out of scope: payroll, recruitment, performance appraisal as an HR process, and employee self-service HR workflows such as clock-in/out or leave application.

1.3 Definitions, Acronyms and Abbreviations

Table 8: Definitions and acronyms

Term	Meaning
TRACE	Task and Resource Attendance and Comprehensive Evaluation System
SRS	Software Requirement Specification
AI / LLM	Artificial Intelligence / Large Language Model
API / REST	Application Programming Interface / Representational State Transfer
JWT	JSON Web Token — a signed token that proves the user's identity

Term	Meaning
ORM	Object-Relational Mapping (SQLAlchemy)
Task Group	A module of work within a project; one bar of the Gantt chart
Project Manager (PM)	An Employee appointed to run one project
Member	An Employee assigned at least one task in a project
Gantt version	One stored, immutable snapshot of a project's Gantt chart
Evaluate Project	The AI action that turns source documents into a plan
MinIO	S3-compatible object store holding uploaded files
Celery	Background job queue used for AI and scheduled work
DFD / ERD	Data Flow Diagram / Entity Relationship Diagram

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- The approved project synopsis of TRACE.
- Official documentation of FastAPI, SQLAlchemy, PostgreSQL, Celery, React and the GitHub REST API (see the Bibliography).

1.5 Overview of the Rest of the SRS

Section 2 gives a general description of the product — its perspective, interfaces, functions, users, constraints and assumptions. Section 3 lists the specific requirements — external interfaces, functional requirements, performance requirements, design constraints, logical database requirements and software system attributes.

2. General Description

2.1 Product Perspective

TRACE is a new, self-contained system. It depends on three external services: **GitHub** (repositories, issues, collaborators and document files), an **OpenAI-compatible AI model** (planning and writing), and an **SMTP mail server** (notification e-mail). Internally it consists of a browser client, an API server, background workers, a relational database, a message broker and an object store.

2.1.1 System Interface. The system is accessed through a web browser on any device with an Internet connection. The browser talks to the API over HTTPS using JSON, and holds one WebSocket connection for live events.

2.1.2 User Interface. The interface is divided into three panels:

- *Landing and Login pages* — a public landing page describing the product, and a login page accepting e-mail and password.

- *Admin panel* (sidebar layout) — Control Panel (CRUD over every table), Workforce (employees, managers, leaves, holidays, attendance, shifts, departments, designations, each with bulk upload), Monitor (service health and recent jobs), Configuration (GitHub and mail settings, each with a connection test).
- *Manager panel* (top bar with a project selector) — a combined dashboard of all the Manager's projects, a create-project wizard, and a project page with tabs Overview, Tree, Gantt, Progress Report and Team.
- *Employee panel* (top bar with a project selector) — a combined dashboard with the Employee's tasks, a project page whose tabs depend on the project role (Project Manager: Overview, Tree, Gantt, Tasks, Project Doc, Team; Member: Overview, Tree, Gantt, My Tasks) and a task page with the progress document and the discussion.
- *Notification bell* on every panel, with the unread count and deep links.

2.1.3 Hardware Interface. Any computer able to run a modern browser; the server needs 2 vCPU, 4 GB RAM and 40 GB disk (Section 1.7).

2.1.4 Software Interface. PostgreSQL (via SQLAlchemy/asyncpg), Redis (Celery broker and publish/subscribe), MinIO (S3 API), GitHub REST API (HTTPS, token authentication), the AI model (OpenAI chat-completions API in JSON mode) and SMTP.

2.1.5 Communication Interface. HTTPS for all client–server traffic, WebSocket (`/api/v1/ws`) for live events, HTTPS to GitHub and the AI provider, SMTP for mail. In production the application is published through a Cloudflare tunnel, so the server opens no inbound port.

2.2 Product Functions

- **Authentication and roles** — login, token refresh, three fixed platform roles and per-project roles.
- **Project management** — create a project (and its GitHub repository), upload source files, appoint and remove Project Managers, edit and delete projects.
- **AI evaluation** — generate the project document, task groups and the first Gantt chart.
- **Task management** — create, edit, assign, re-assign and delete tasks, each mirrored as a GitHub issue; set status under the status-authority rule.
- **AI assistance** — suggest Project Managers, assignees and tasks; write task progress documents; revise the plan when the project document changes; write the weekly progress report.
- **Visualisation** — project tree graph and versioned Gantt chart.

- **Communication** — notifications (in-app, live and e-mail) and per-task discussions.
- **Reporting** — AI progress and report documents; CSV report download for a date range.
- **Administration** — table CRUD, workforce records with bulk upload, service monitoring, GitHub and mail configuration.

2.3 User Characteristics

Table 9: User classes

User	Characteristics
Administrator	Operates the platform. Technically skilled; maintains data, configuration and the workforce records; does not take part in project delivery.
Manager	Brings projects in. Needs no technical skill beyond uploading documents and reading reports; creates projects, appoints Project Managers and watches progress.
Employee as Project Manager	Plans and runs a project: edits the project document, creates and assigns tasks, sets any task's status. Familiar with software delivery.
Employee as Member	Does the work: sees the project, works on their own tasks on GitHub, discusses them and keeps their status current, including marking them complete.

2.4 General Constraints

- The system runs in current versions of Chrome, Firefox and Edge; it is designed for desktop screens.
- A GitHub Organization and a fine-grained token are required before any project can be created (a project's repository is created with it).
- AI features require an API key configured on the server; without it, AI actions are refused with a clear message and the rest of the system keeps working.
- Every AI call must run in the background; no web request may wait for the model.
- Repositories created by TRACE are public; confidential material must not be placed in requirement documents.

2.5 Assumptions and Dependencies

- GitHub, the AI provider and the mail server are reachable from the server.
- Each Employee who works on tasks has a GitHub account and records its username in their profile, and accepts the repository invitation GitHub e-mails them.
- The Celery worker and beat processes are running; otherwise background work waits in the queue.

- The AI model can return JSON output on request (OpenAI-compatible "JSON mode").

3. Specific Requirements

3.1 External Interface Requirements

The system provides a consistent graphical interface built from one component library, in light and dark themes. All data exchange with the server uses JSON over HTTPS; uploads use multipart form data; the report download is a CSV file. The REST API is versioned under `/api/v1` and documented automatically (OpenAPI) at `/docs`.

3.2 Functional Requirements

Table 10: Functional requirements

Id	Requirement	Actor
FR-01	The system shall authenticate users by e-mail and password and issue a JWT access token and a refresh token.	All users
FR-02	The system shall support exactly three platform roles — Admin, Manager, Employee — and route each to its own panel; a user opening another role's panel shall see a 403 page.	System
FR-03	Public self-registration shall only create Employee accounts; Admin and Manager accounts shall be created by an Admin.	System / Admin
FR-04	A Manager shall create a project with a name and description; the system shall create the project's public GitHub repository under the configured Organization in the same operation, and refuse the project if the repository cannot be created.	Manager
FR-05	A Manager shall upload one or more source files (PDF, DOCX, MD, TXT, up to 20 MB each) to a project; the system shall store them in MinIO, extract their text and report rejected files.	Manager
FR-06	A Manager shall press *Evaluate Project*; the system shall, in the background, write the project document to the repository (twice: initial and living copy), create task groups with durations and dependencies, and save Gantt version 1.	Manager / AI
FR-07	Evaluation shall be refused while running, without extracted text, or once the project has tasks; a failed evaluation shall be shown with the step that failed and shall keep no partial data.	System
FR-08	A Manager shall appoint and remove Project Managers from the active Employees, optionally from an AI-ranked list with score and reasoning; the appointee shall be invited to the repository.	Manager / AI
FR-09	A Project Manager shall create, edit and delete task groups; a group holding tasks shall not be deleted.	Project Manager
FR-10	A Project Manager shall create a task in a group; the system shall create the matching GitHub issue in the same request and roll the task back if the issue cannot be created.	Project Manager

Id	Requirement	Actor
FR-11	A Project Manager shall assign or re-assign a task to an Employee; the system shall make them a Member, invite them to the repository, assign the GitHub issue and notify them.	Project Manager
FR-12	The system shall suggest tasks for a task group and rank assignees for a draft task using AI; suggestions shall only fill the form, never create or assign.	Project Manager / AI
FR-13	After a task is created, the system shall write its step-by-step plan and checklist (task progress document) to the repository in the background.	AI
FR-14	The assignee shall set their own task's status to Pending, In Progress, In Review, Blocked or Completed; the Project Manager shall set any status except Overdue; everyone else shall be refused.	Member / Project Manager
FR-15	Completing or cancelling a task shall close its GitHub issue; re-opening shall re-open it; deleting shall close it as "not planned".	System
FR-16	Each task group's completion percentage and status shall be recomputed from its tasks on every change.	System
FR-17	The system shall regenerate the Gantt chart as a new version when a task is completed or re-opened, when tasks become overdue, when the project document changes, and on the Project Manager's request; bursts of triggers shall produce one version.	System / AI
FR-18	A Project Manager shall edit the living project document; a changed document shall trigger a re-plan in which the AI may revise durations and add groups.	Project Manager / AI
FR-19	The system shall show a project tree graph (Project → Task Groups → Tasks → assignees) and a Gantt chart with version history to everyone who can see the project.	All project users
FR-20	Every Monday the system shall rewrite each active project's progress document and report document with AI; the owner or a Project Manager may request it at any time.	AI / Manager / PM
FR-21	A nightly job shall mark open past-due tasks Overdue and notify; a daily job shall remind about tasks due tomorrow.	System
FR-22	The system shall notify the assignee on assignment; the Project Managers on completion (and the assignee if someone else completed it); the owner when evaluation finishes or fails — in-app, live and by e-mail.	System
FR-23	Each task shall have a discussion thread visible to its assignee, its assigner, the project's Project Managers and the owning Manager, with one level of replies.	Project users
FR-24	Managers and Employees shall download a CSV report of the project → task group → task tree for a date range.	Manager / Employee
FR-25	The Admin shall perform CRUD on every table except the secret settings table through the Control Panel.	Admin
FR-26	The Admin shall maintain employees, managers, leaves, holidays, attendance, shifts, departments and designations, each with CSV bulk upload that adds good rows and reports rejected ones.	Admin

Id	Requirement	Actor
FR-27	The Admin shall configure the GitHub Organization, token and committer, and the mail server, and test each connection; secrets shall be write-only.	Admin
FR-28	The Admin shall monitor the health of the API, database, Redis, Celery worker and beat, MinIO, mail, WebSocket and AI, and see recent job outcomes.	Admin
FR-29	Deleting a project shall be refused while it has open tasks and shall archive its repository first.	Manager
FR-30	Every project and task endpoint shall check the caller's project role in addition to the platform role.	System

3.3 Performance Requirements

- Standard API requests shall respond within 2 seconds; AI actions return a job id immediately and never block a request.
- Dashboard pages shall load within 4 seconds on a broadband connection.
- A burst of status changes shall produce a single Gantt re-plan (30-second debounce).
- Live notifications shall reach an open browser within a few seconds of the event.
- The system shall support at least 200 concurrent users on the recommended server.

3.4 Design Constraints

- The frontend uses React with the shadcn/ui component library, Tailwind design tokens and lucide-react icons only.
- The backend uses FastAPI with asynchronous SQLAlchemy; the database schema is created and changed only by Alembic migrations.
- Every AI call runs in a Celery job and every AI response is validated against a declared JSON schema.
- All GitHub calls go through one service module; all credentials are resolved through one settings function.
- One configuration file (`.env`) holds all environment settings.

3.5 Logical Database Requirements

The data is stored in a relational PostgreSQL database. Every table has a surrogate `BIGINT` primary key, `created_at` / `updated_at` timestamps and a `deleted_at` column for soft deletion. Foreign keys enforce every relationship; unique constraints protect e-mails, usernames, role names, (project, user) assignments and (category, key) settings. Generated documents are stored as GitHub file URLs, never as text in the database. The database design is given in Chapter 3.

3.6 Software System Attributes

3.6.1 Reliability. A task and its GitHub issue are created together or not at all; a failed AI job leaves a visible error and no partial data; every Gantt version is validated before it is saved. The system produces correct output for valid input and meaningful error messages for invalid input.

3.6.2 Availability. The system is designed to be available around the clock. Background work is queued in Redis, so a restarted worker continues where it stopped. Health checks for every service are visible on the Admin Monitor page.

3.6.3 Security.

- Passwords are hashed with bcrypt (12 rounds); plain passwords are never stored.
- Every request is authenticated with a signed JWT; every route declares a role guard, and every project route also checks the project role.
- Stored secrets (GitHub token, SMTP password) are encrypted with Fernet and are never returned by any API.
- All input is validated server-side with Pydantic schemas; SQL is issued only through the ORM with bound parameters.
- Production refuses to start with default passwords, weak JWT secrets or a missing encryption key.

3.6.4 Maintainability. The code follows fixed conventions (one service per concern, one way to call the AI, one way to call GitHub). Every change ships with automated tests, including an audit that fails if any route lacks an authorisation guard and a test that builds the whole schema from migrations. The design, decisions and status of every feature are documented alongside the code.

3.6.5 Portability. The client runs in any modern browser on any operating system. The supporting services run in Docker containers, and the application runs on any system with Python and Node.js.

2.4 Software Engineering Paradigm Applied

TRACE was developed using the **Iterative and Incremental model**, organised as a sequence of numbered build phases, each of which delivers a working, tested increment.

Why this paradigm. A pure Waterfall model was unsuitable because the requirements changed materially during development: after the first increments the core of the product moved from attendance to AI-driven project delivery, and later decisions (for example making task completion manual instead of CI/CD-driven, or fixing the AI configuration in the environment file) changed parts of the design. A prototyping model alone would not have produced a maintainable system. The iterative and incremental

model allowed the requirements to be revised between increments while every increment was still specified, built, tested and documented properly.

How it was applied.

1. *Analysis and design* produced the specification (roles, panels, schema, AI pipeline) and a dependency graph of build phases.
2. Each *increment* (phase) had written steps and a "done when" condition — for example, *"a Manager can create a project and assign a Project Manager, and that Project Manager can create tasks that appear as GitHub issues, with no AI involved at all."*
3. Each increment was *tested* with automated tests before the next began; the test suite grew with every phase and was always run in full.
4. After each increment the *documentation was updated in the same change*, and any decision that was not obvious was recorded with its reason.
5. When requirements changed, the *specification was changed first*, then the code.

Table 11: Build increments

Phase	Increment	Delivered
0	Clear the ground	Removal of retired features (clock-in, geo-fencing, dynamic roles, group chat)
1	Schema	New project tables, backfill migrations, three roles
2	Panels	Three panel layouts, route guards, landing page, 403 page
3	Projects without AI	Projects, source files, task groups, tasks as GitHub issues, status authority
3a	GitHub owner model	Repository creation, collaborator invitations, encrypted settings
4	Visualisation	Tree graph and versioned Gantt chart
5	AI pipeline	Evaluation, suggestions, task documents, re-planning, weekly reports
7	Notifications and discussion	Notification centre, e-mail, live events, discussions
8	Administration	Mail configuration, Workforce, Monitor, report download
9	Polish	Pagination, search, empty and error states, revision sweep

(Phase 6, a CI/CD-driven automatic completion feature, was removed from the design before it was built; the phase numbers were kept so that existing references stayed valid.)

2.5 Data Models and Analysis Diagrams

2.5.1 Data Flow Diagrams (DFD)

A Data Flow Diagram shows how data moves between external entities, processes and data stores. The context diagram (Level 0) shows TRACE as a single process; Level 1 breaks it into its main processes; Level 2 refines the two most important processes — project evaluation and task management.

Notation: rectangles are external entities, circles (rounded boxes) are processes, open-ended boxes are data stores, and arrows are data flows.

Level 0 DFD (Context Diagram)

The external entities are the three users (Admin, Manager, Employee) and the three external systems (GitHub, the AI model and the mail server).

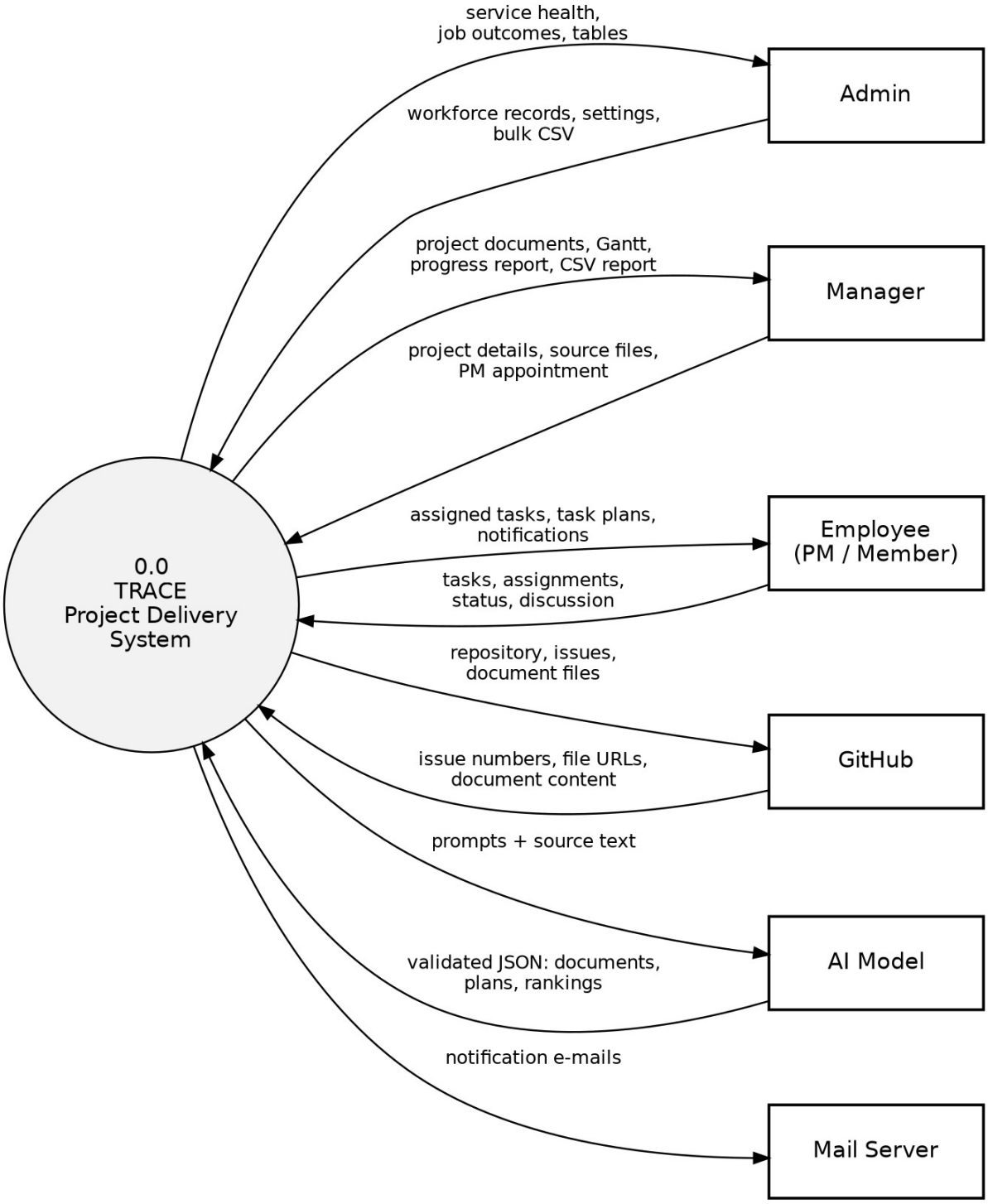


Figure 3: Level 0 DFD — context diagram

Level 1 DFD

Table 12: Level 1 DFD processes

No.	Process	Description
1.0	Authenticate user	Checks credentials, issues tokens, resolves the platform role
2.0	Manage projects	Creates projects and repositories, stores source files, manages the Project Manager roster

No.	Process	Description
3.0	Evaluate project	Runs the AI pipeline: project document, task groups, first Gantt version
4.0	Manage tasks	Task groups, tasks, GitHub issues, assignment, status, progress documents
5.0	Plan and report	Gantt re-planning, weekly progress and report documents, CSV report
6.0	Notify and discuss	Notifications, e-mail, live events, task discussions
7.0	Administer	Workforce records, table CRUD, configuration, monitoring

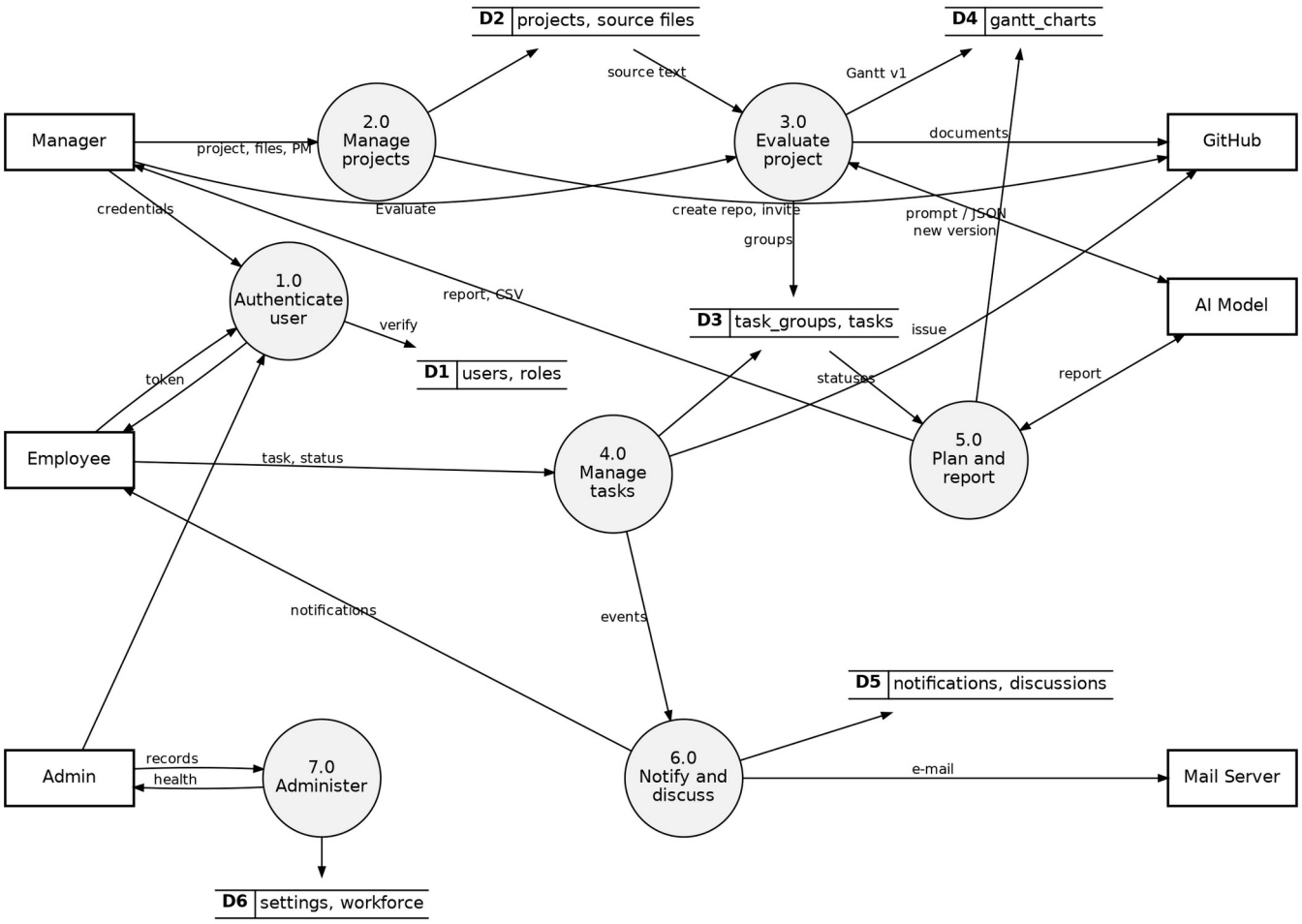


Figure 4: Level 1 DFD

Level 2 DFD — Process 3.0 Evaluate Project

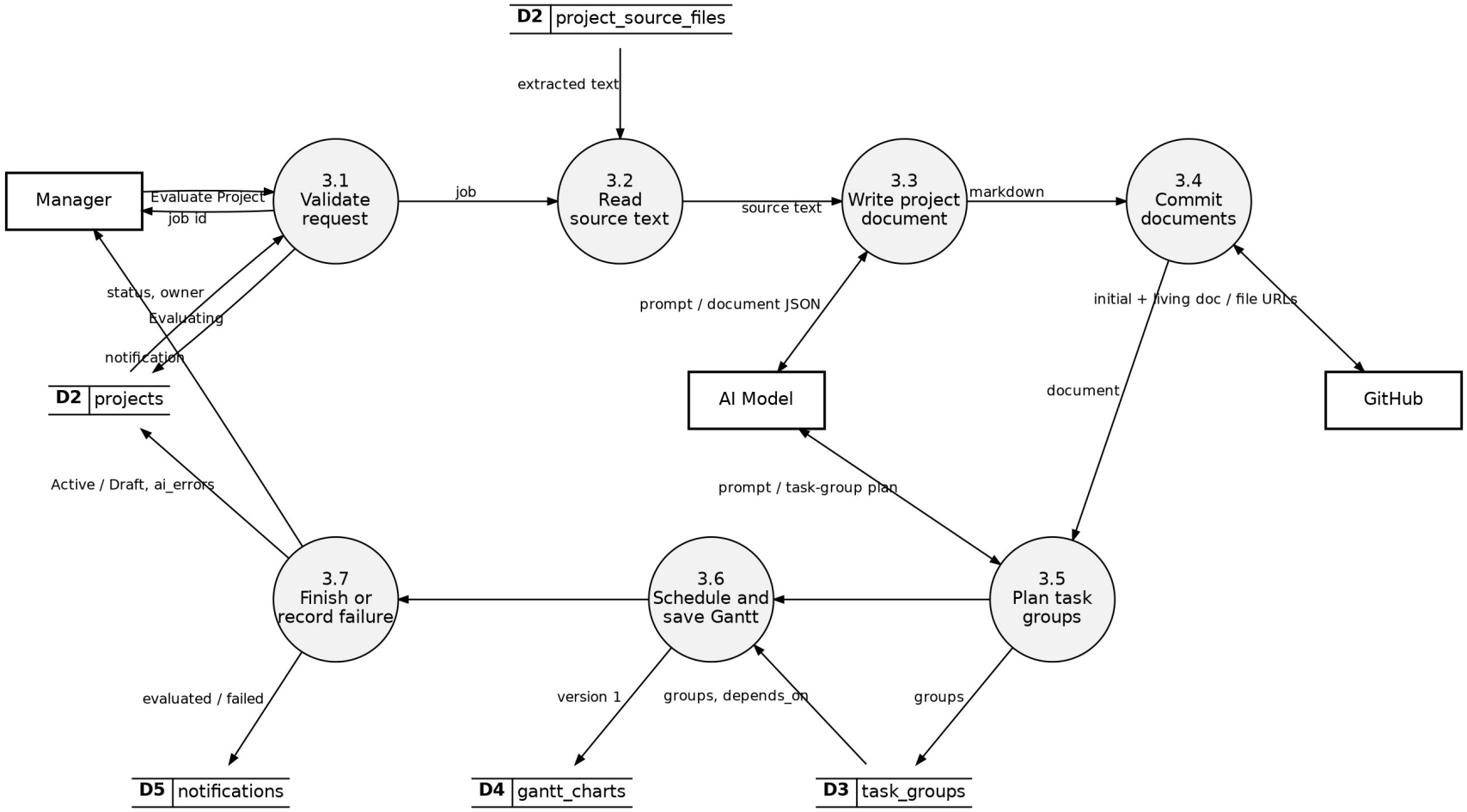


Figure 5: Level 2 DFD — Evaluate Project (process 3.0)

Level 2 DFD — Process 4.0 Manage Tasks

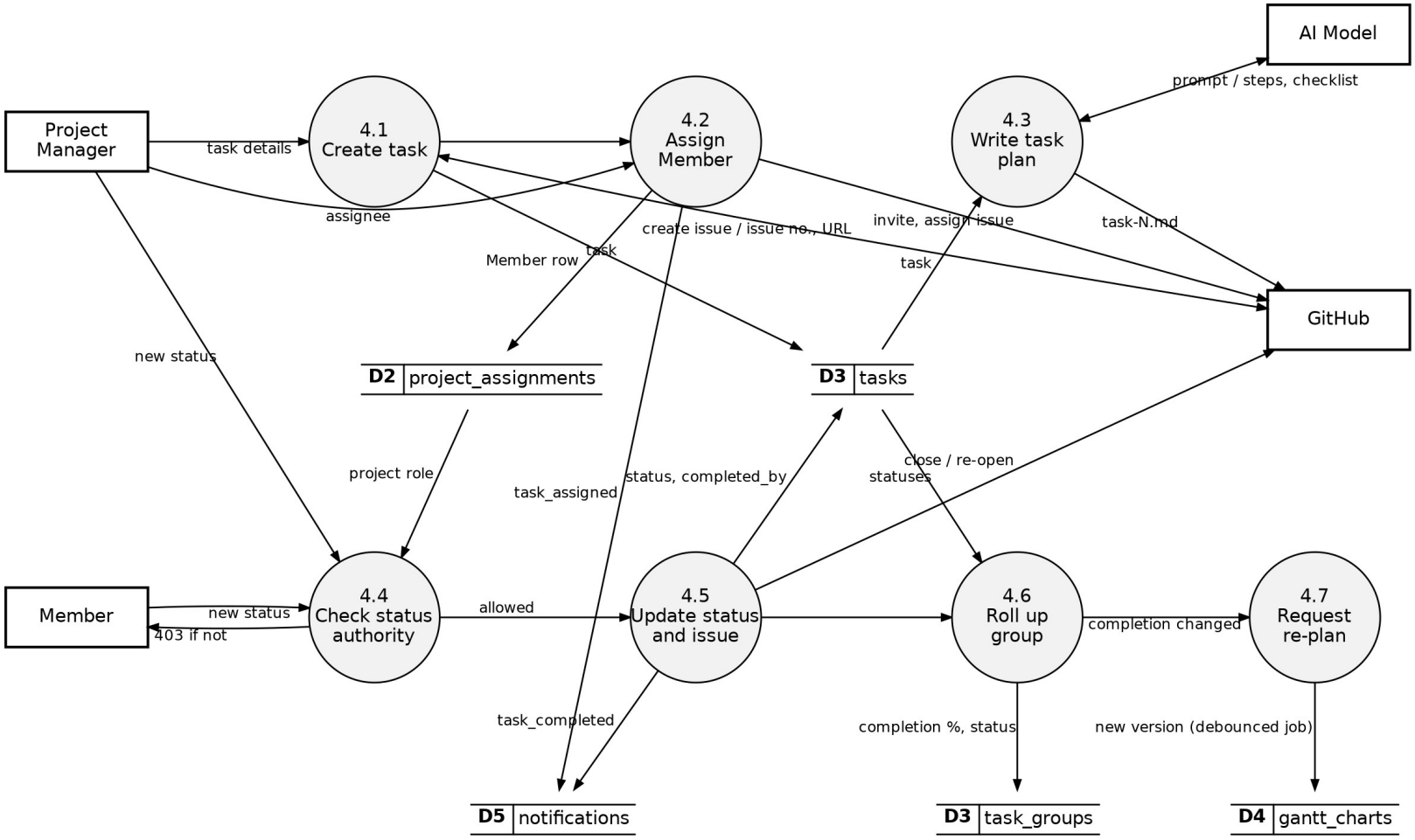


Figure 6: Level 2 DFD — Manage Tasks (process 4.0)

2.5.2 Control Flow Diagram

The control flow diagram shows how control passes through the layers of the system for every request: the browser sends a request with its token; the API authenticates it, checks the platform role on the route and the project role in the service; the service validates the input, performs the change and, where needed, calls GitHub before committing; after the commit, background jobs are queued and live events published. AI work never runs inside the request — the route returns a job id, and the browser polls for the result.

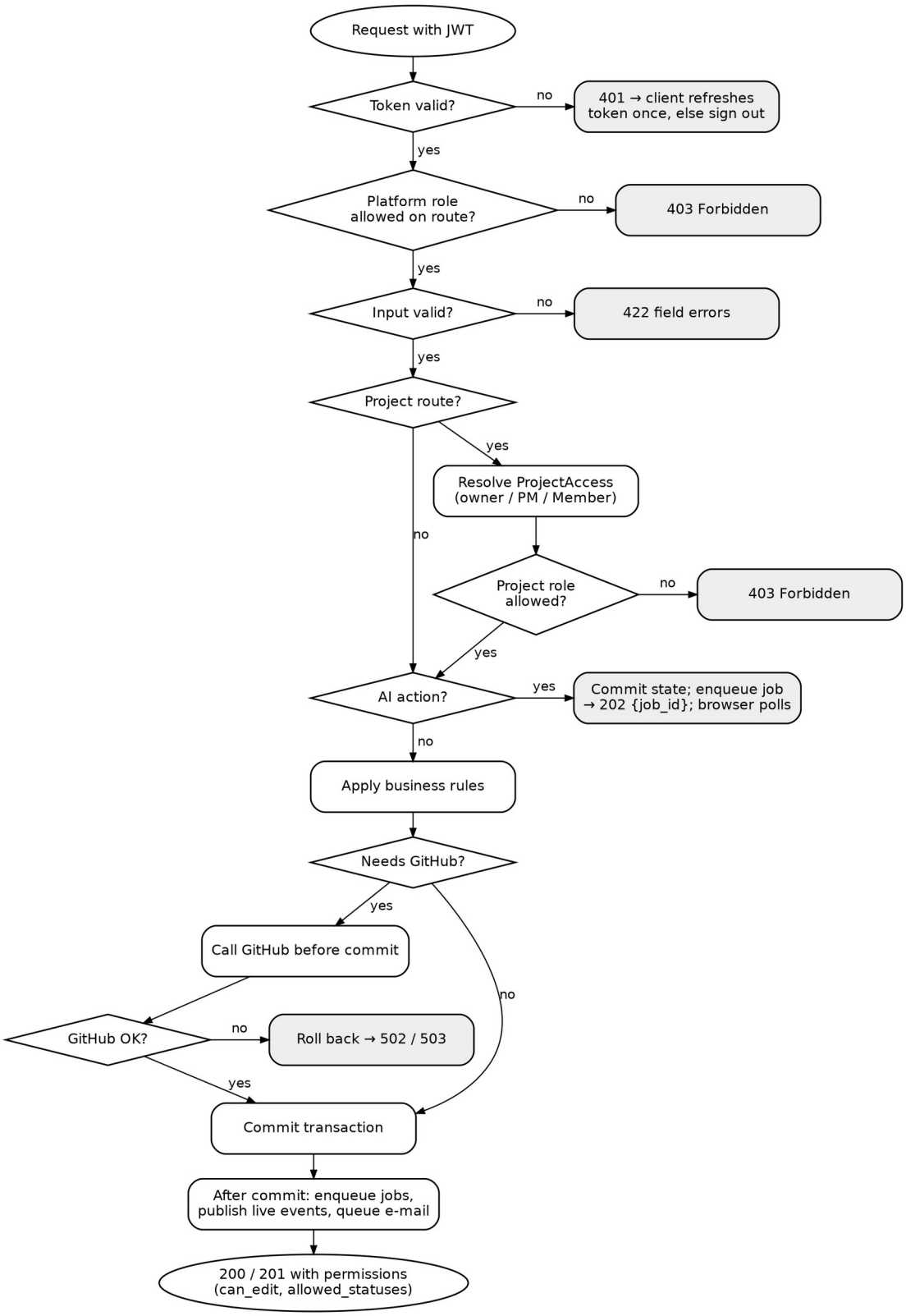


Figure 7: Control flow of a request through TRACE

2.5.3 State Diagrams

State diagrams show the states an object can be in and the events that move it between them.

Task

A task starts *Pending*. The assignee moves it through *In Progress*, *In Review* and *Blocked* and marks it *Completed*; the Project Manager can set any of these and can also *Cancel* it. The nightly sweep moves open past-due tasks to *Overdue*. Completing or cancelling closes the GitHub issue; any move back to an open state re-opens it.

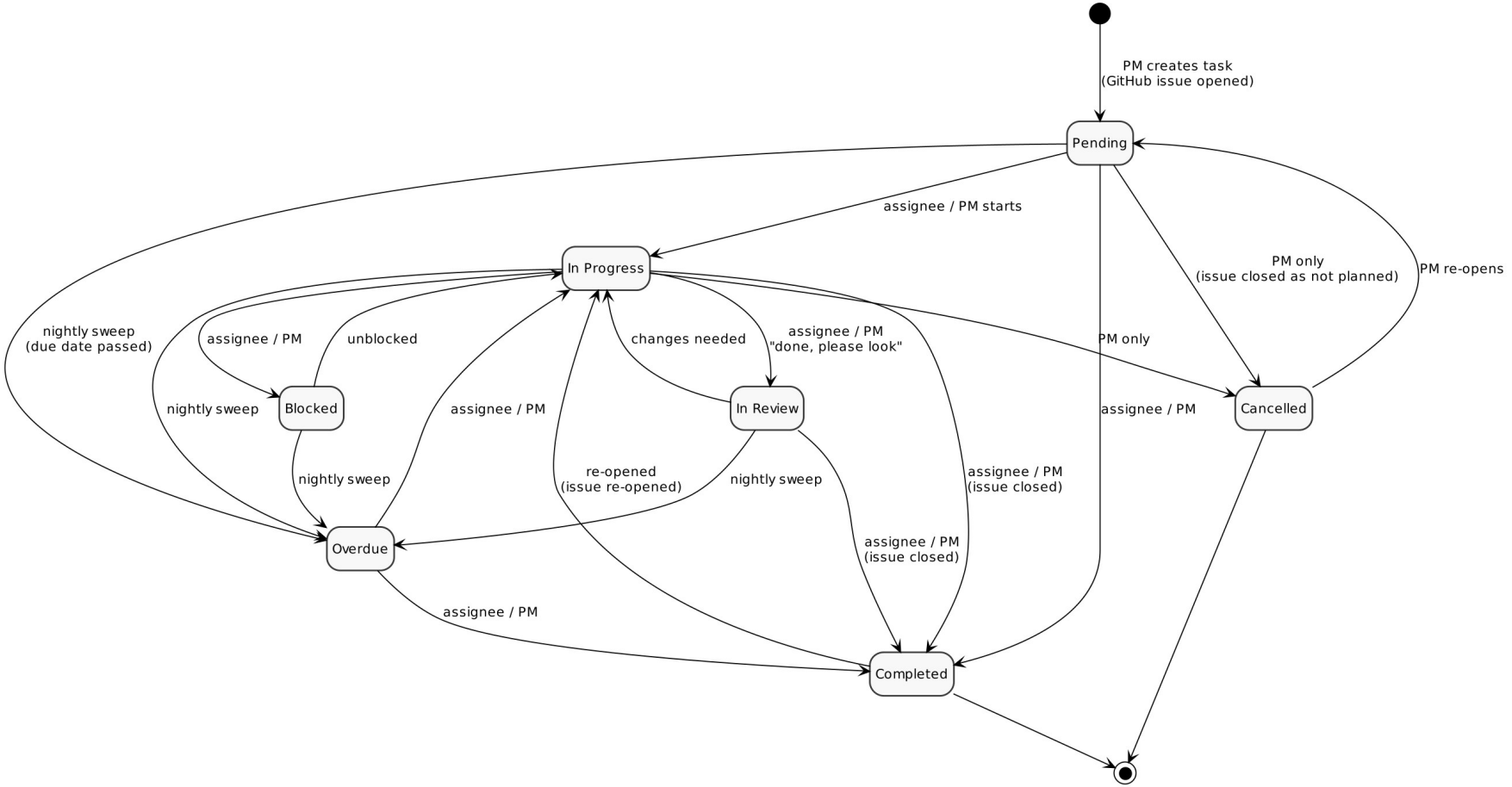


Figure 8: State diagram — Task

Project

A project is created as *Draft*. Pressing *Evaluate Project* moves it to *Evaluating*; success makes it *Active*; failure returns it to *Draft* (or *Active* if it had been evaluated before). An active project can be marked *Completed* or *Archived* by its owner.

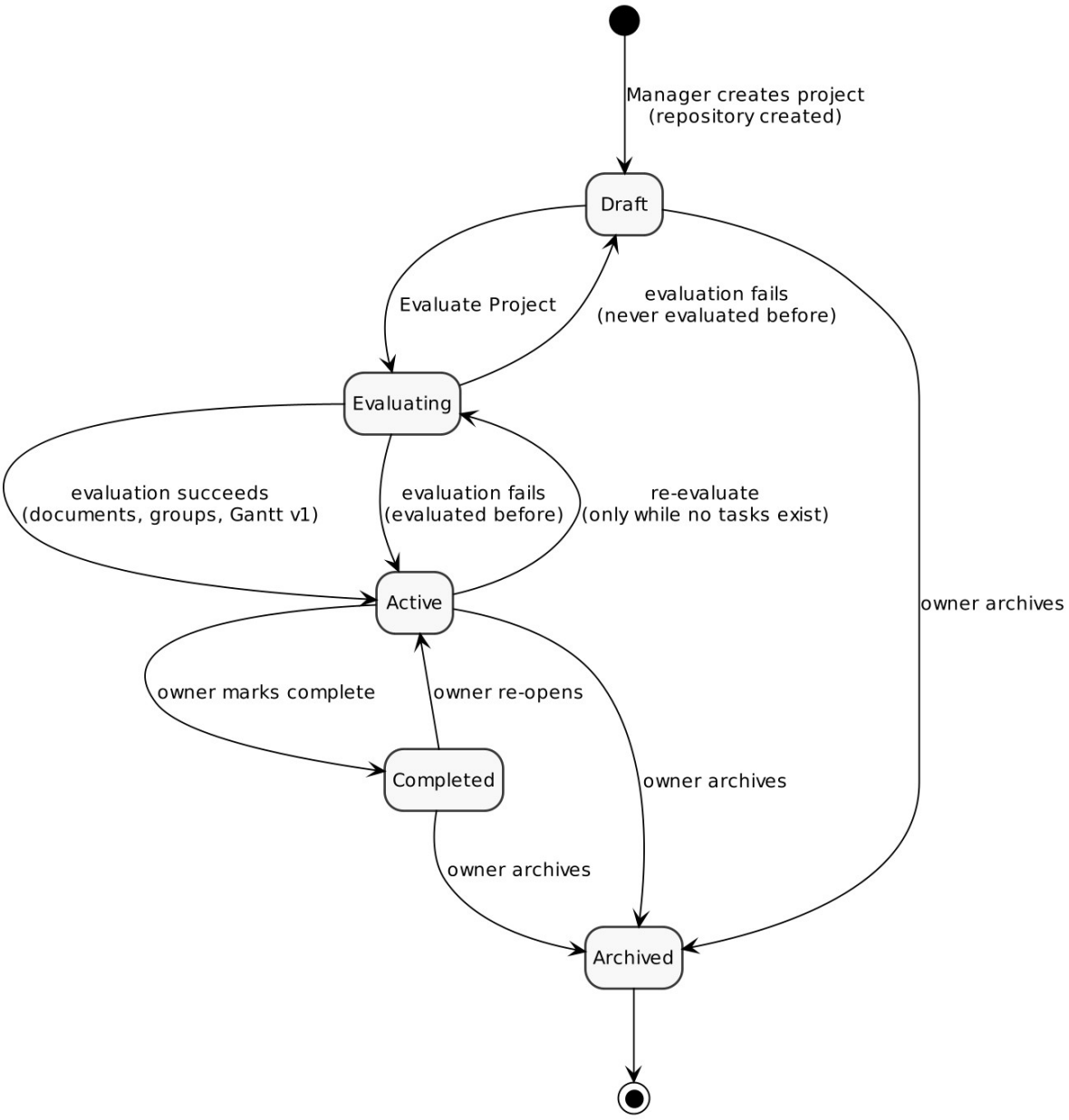


Figure 9: State diagram — Project

Task Group

A task group's status is rolled up from its (non-cancelled) tasks after every change. It is *Pending* while no task has started; it becomes *In Progress* as soon as any task is In Progress, In Review or Completed (the actual start date is stamped the first time); it becomes *Completed* when every task is completed (the actual end date is stamped) and

returns to *In Progress* if one is re-opened; it shows *Blocked* when every task in it is blocked.

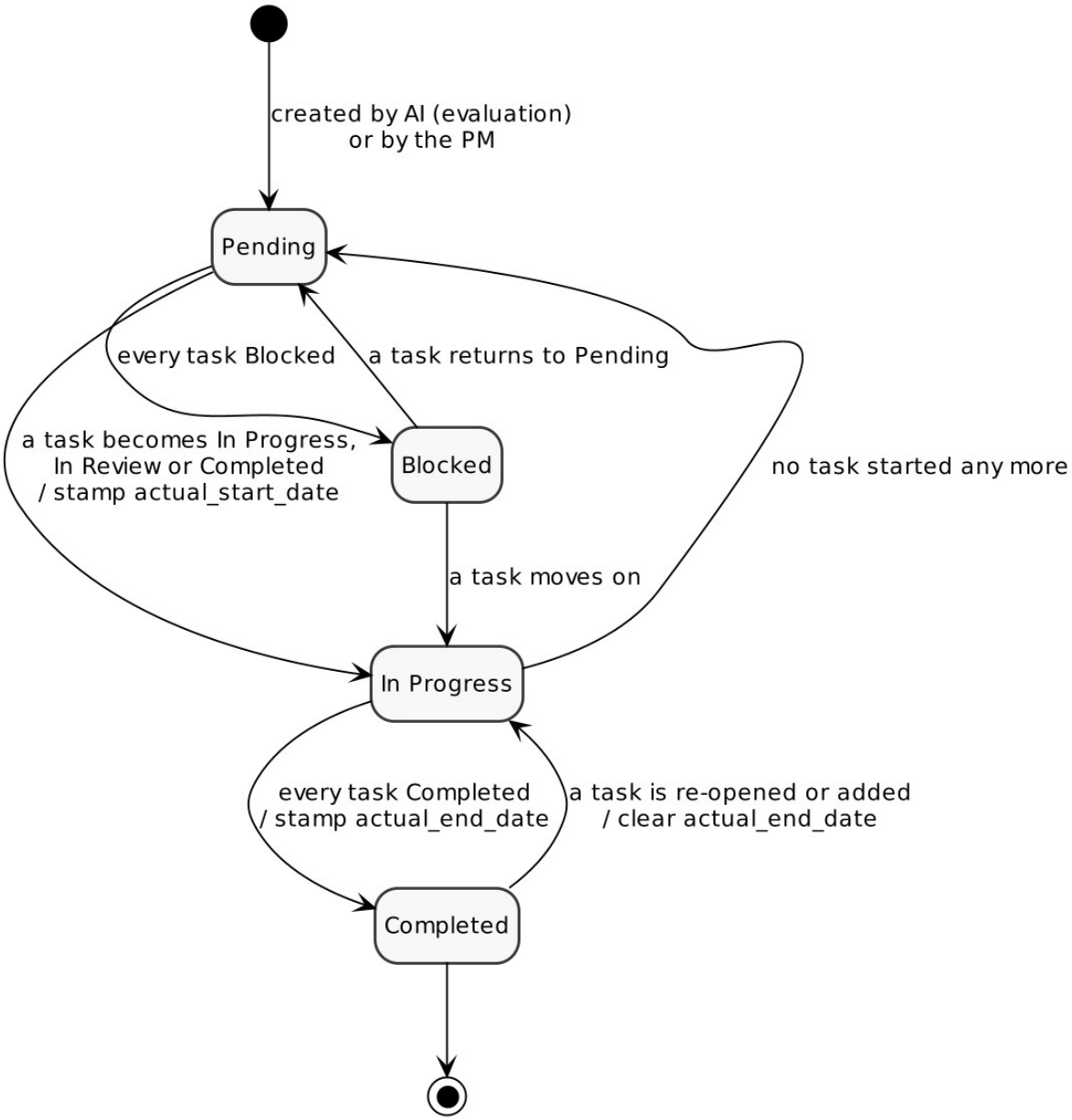


Figure 10: State diagram — Task Group

2.5.4 Sequence Diagrams

Evaluate Project

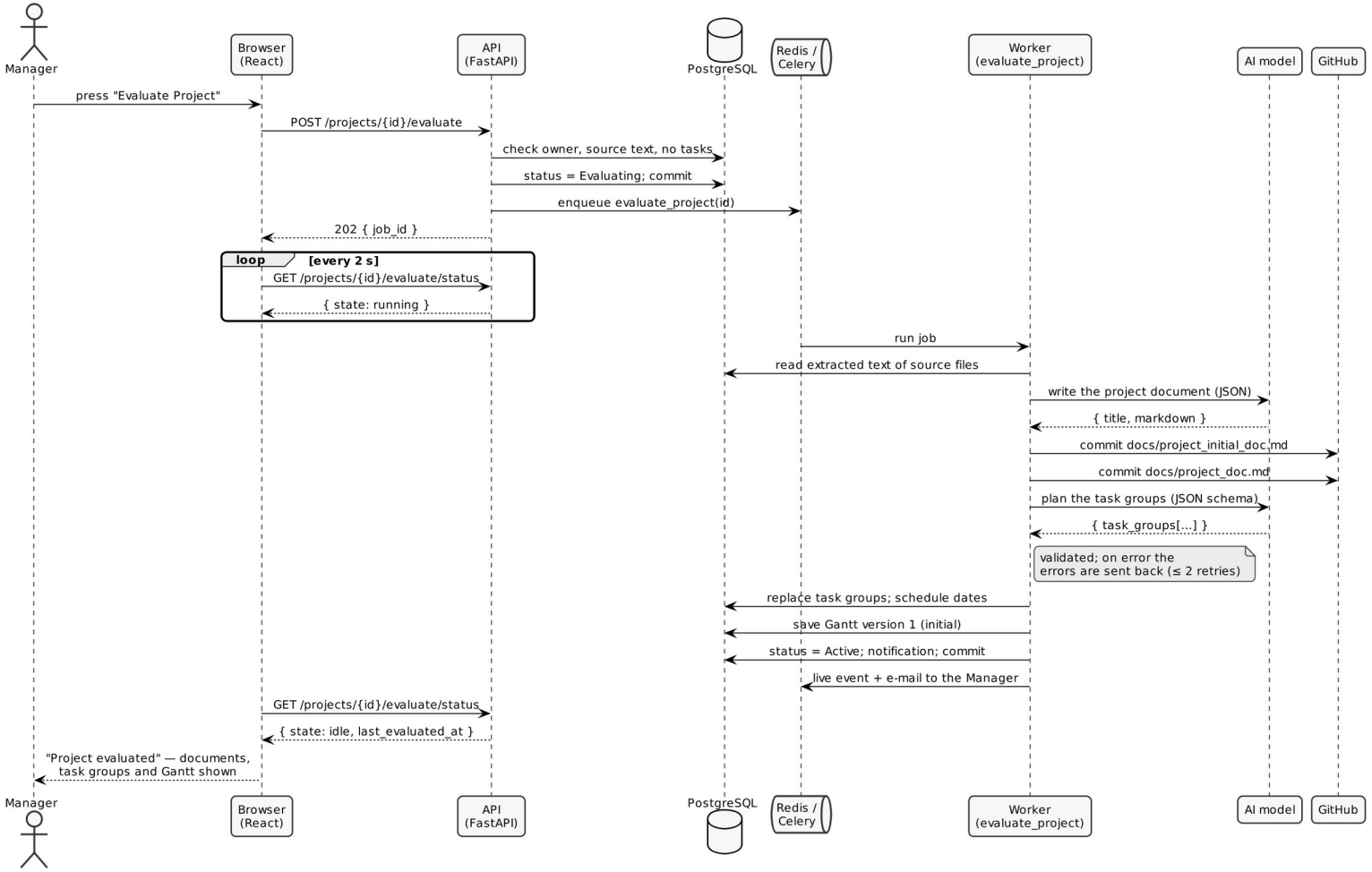


Figure 11: Sequence diagram — Evaluate Project

Create and complete a task

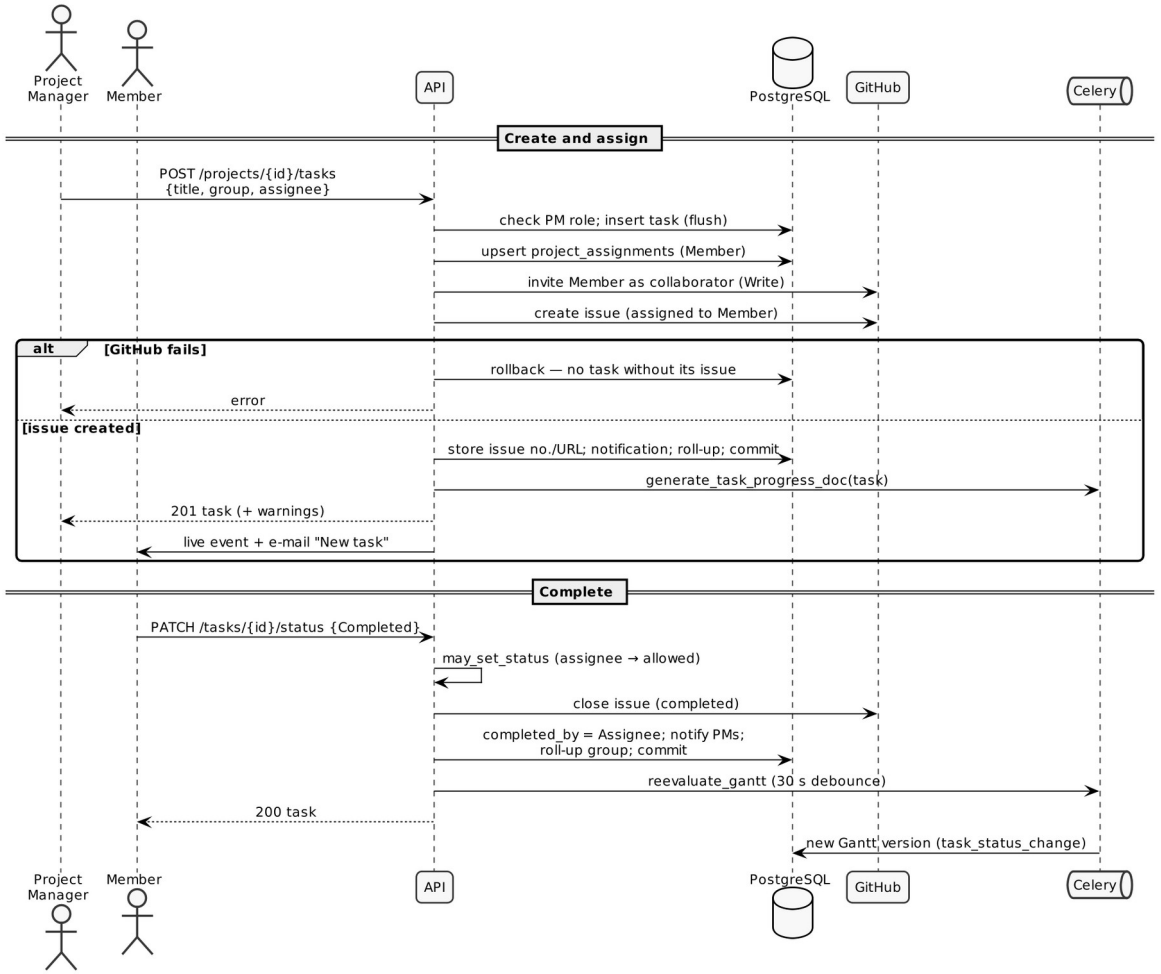


Figure 12: Sequence diagram — creating and completing a task

2.5.5 Entity Relationship Model

The ER model shows the main entities of TRACE and the relationships between them. Attributes are listed in full in the database design (Section 3.4).

Table 13: Relationships between the main entities

Entity	Entity	Cardinality	Relationship
User	Role	M : N (through user_roles)	holds
User	Employee	1 : 1	is recorded as
User (Manager)	Project	1 : N	owns
Project	Project Source File	1 : N	has
Project	User	M : N (through project_assignments)	staffs
Project	Task Group	1 : N	is divided into
Task Group	Task	1 : N	contains
Task Group	Task Group	M : N (depends_on)	waits for
User	Task	1 : N	is assigned
Task	Task Discussion	1 : N	is discussed in
Task Discussion	Task Discussion	1 : N	is replied to by
Project	Gantt Chart	1 : N (versions)	is scheduled by
User	Notification	1 : N	receives
Employee	Productivity Score	1 : N	is scored by

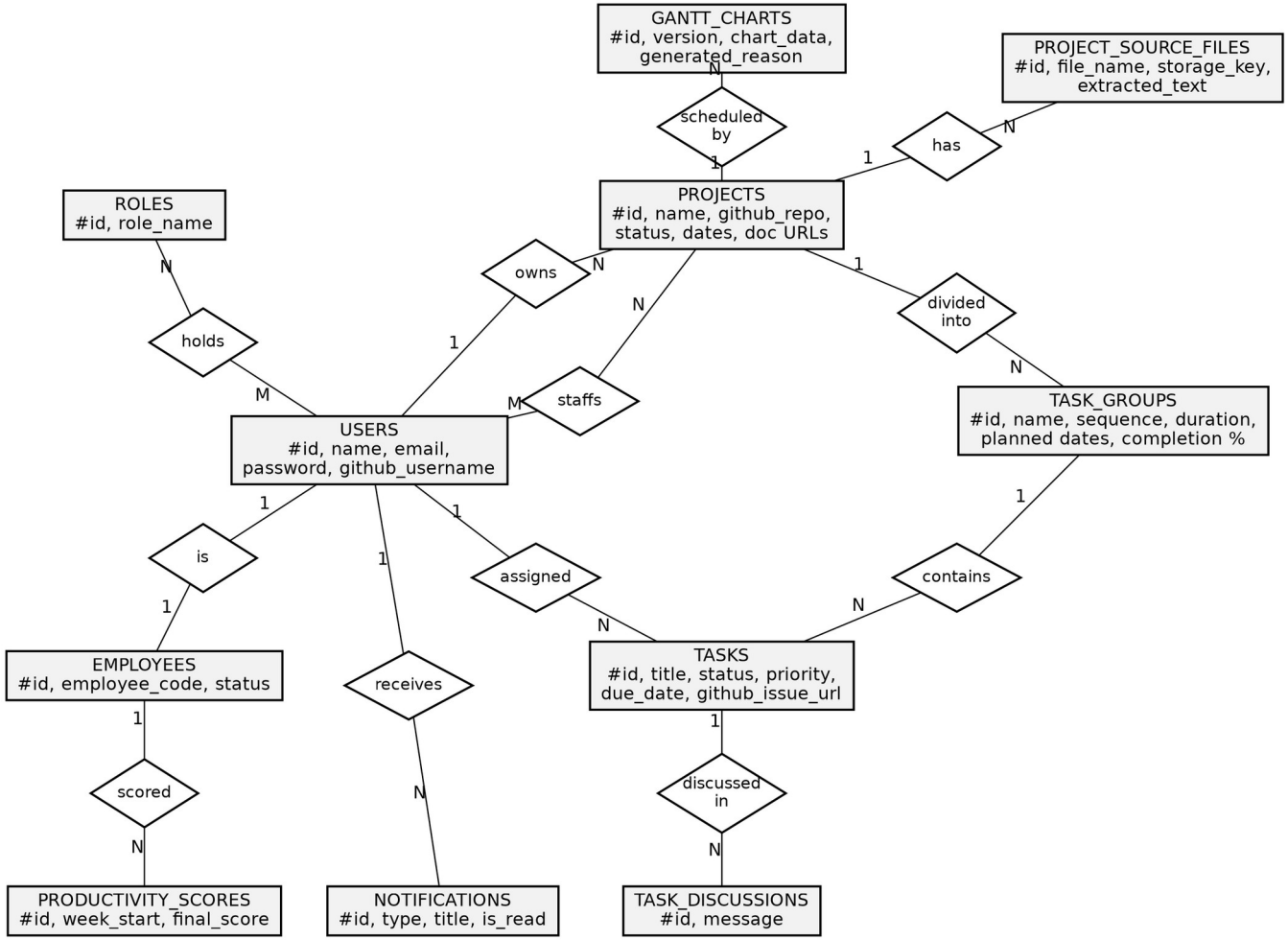


Figure 13: Entity Relationship Diagram

2.5.6 Class Diagram

The class diagram shows the main classes of the backend: the ORM entity classes with their attributes and relationships, and the service and access-control classes that operate on them.

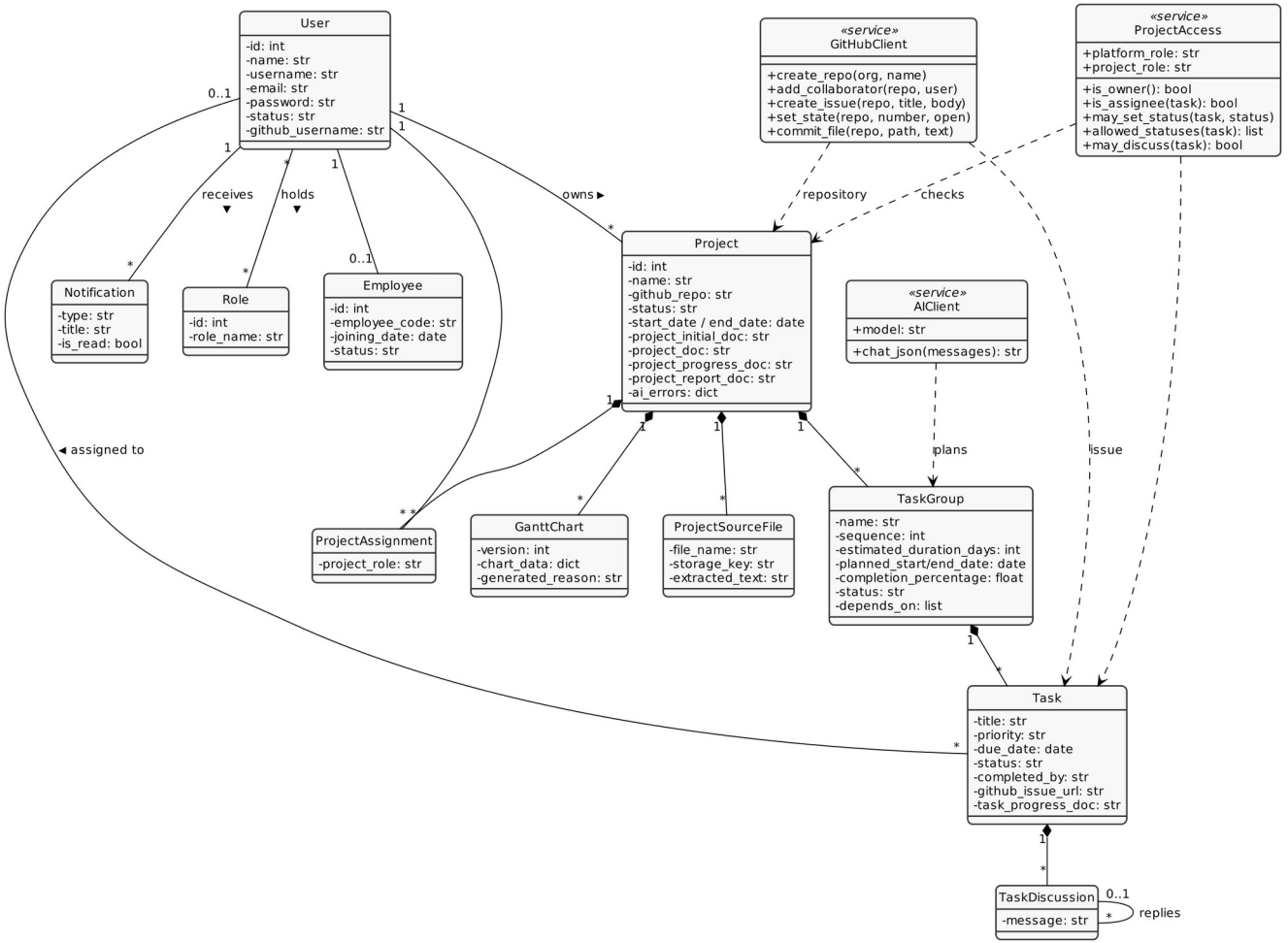


Figure 14: Class diagram

2.5.7 Use Case Diagram

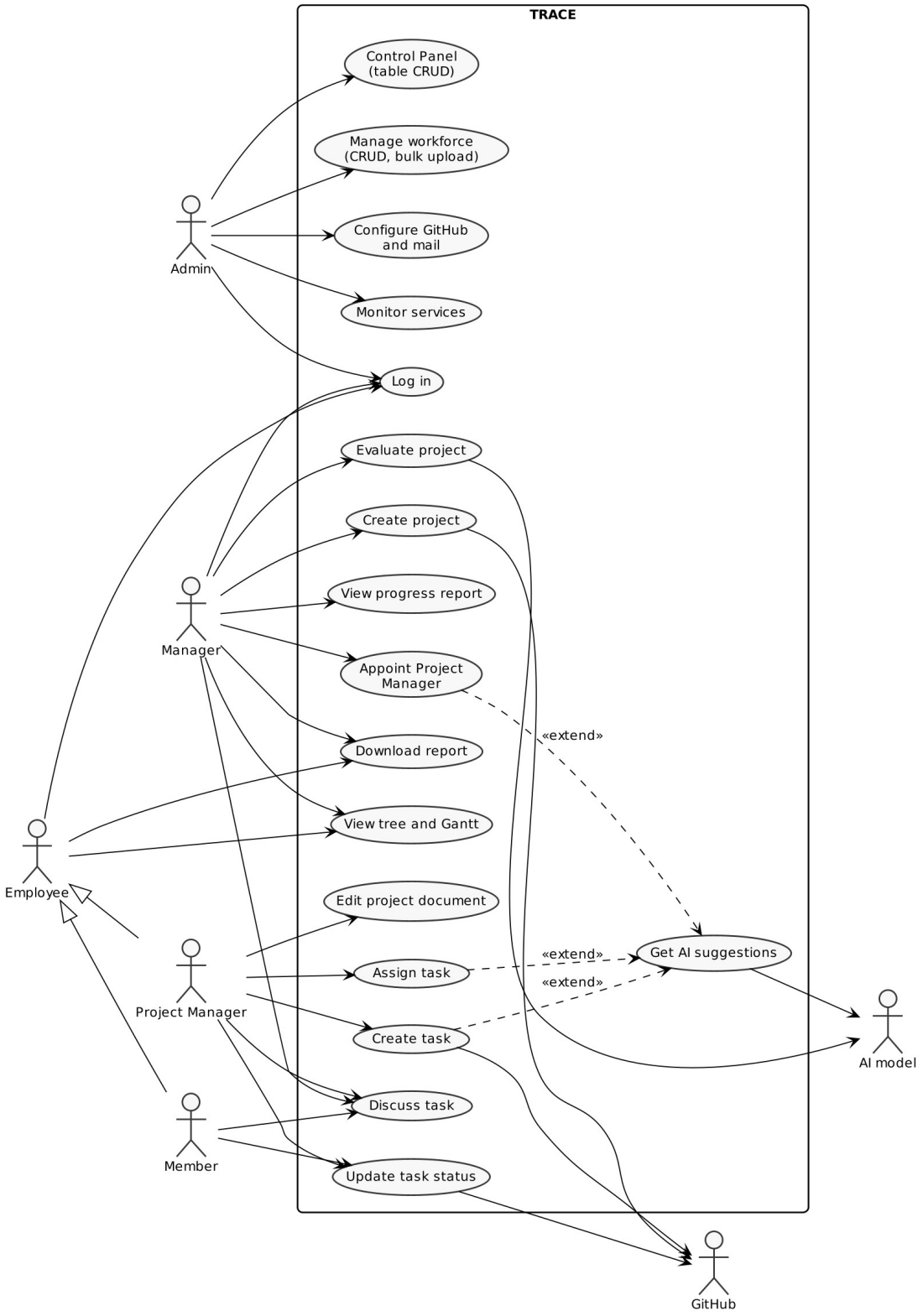


Figure 15: Use case diagram

Table 14: Principal use cases

Id	Use case	Actor	Summary
UC-01	Log in	All	Enter e-mail and password; land on the role's panel
UC-02	Create project	Manager	Name, description, source files; repository created
UC-03	Evaluate project	Manager	AI writes the project document, task groups, Gantt v1
UC-04	Appoint Project Manager	Manager	Pick an Employee, optionally from AI suggestions
UC-05	View progress report	Manager	Read the AI-written progress and report documents
UC-06	Edit project document	Project Manager	Save the living document; the Gantt re-plans
UC-07	Create task	Project Manager	Task and GitHub issue created; AI may suggest content and assignee
UC-08	Assign task	Project Manager	Member invited and notified; issue assigned
UC-09	Update task status	Member / PM	Server enforces who may set what; completion closes the issue
UC-10	Discuss task	Assignee, assigner, PMs, Manager	Post and reply in the task's thread
UC-11	View tree and Gantt	All project users	Graph of the project; Gantt with version history
UC-12	Download report	Manager / Employee	CSV of the project tree for a date range
UC-13	Manage workforce	Admin	CRUD and bulk upload of workforce records
UC-14	Configure GitHub and mail	Admin	Save settings; test the connection
UC-15	Monitor services	Admin	Health of every service and recent jobs

2.5.8 Activity Diagrams

Evaluate Project

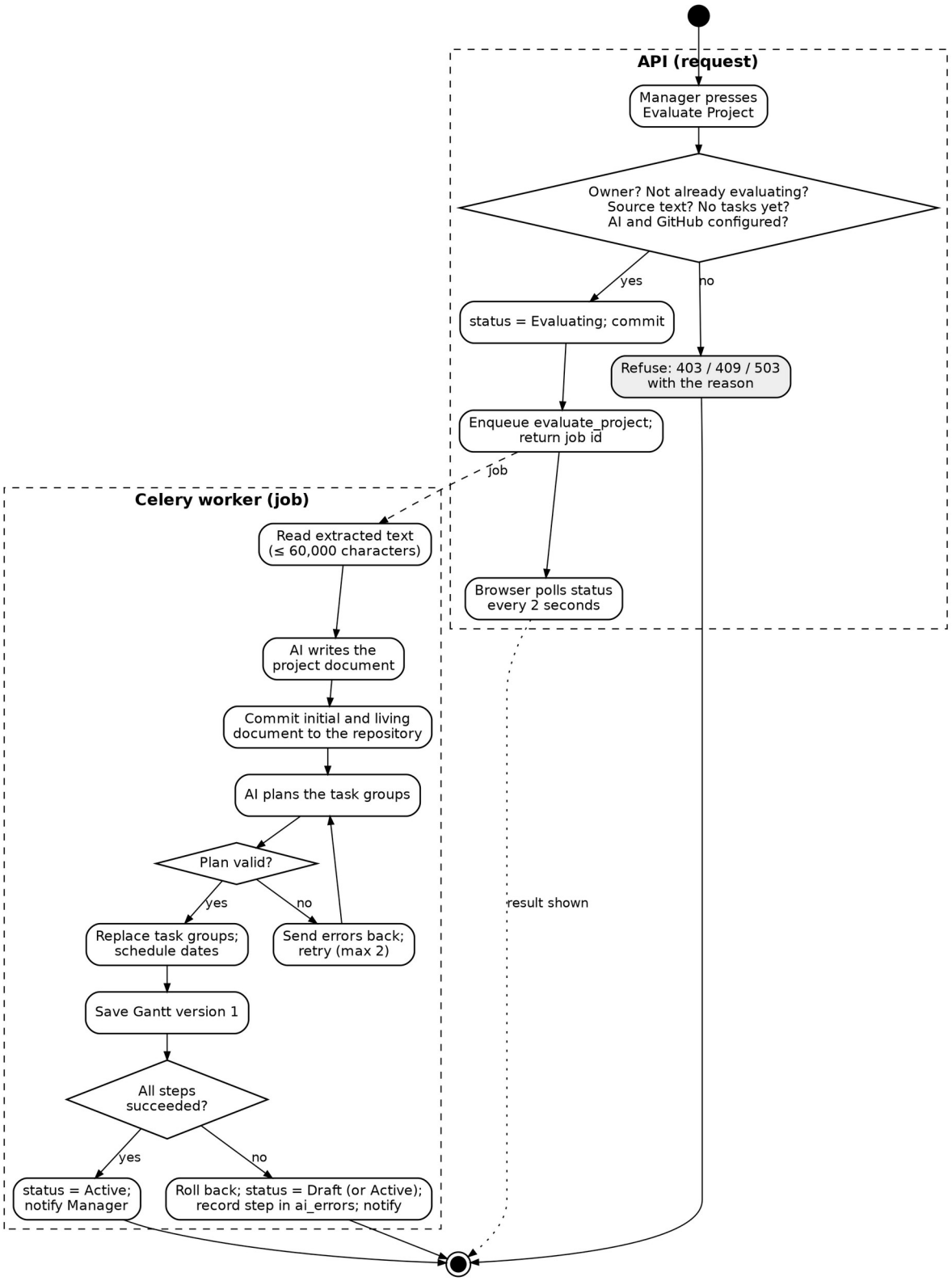


Figure 16: Activity diagram — Evaluate Project

Task life cycle

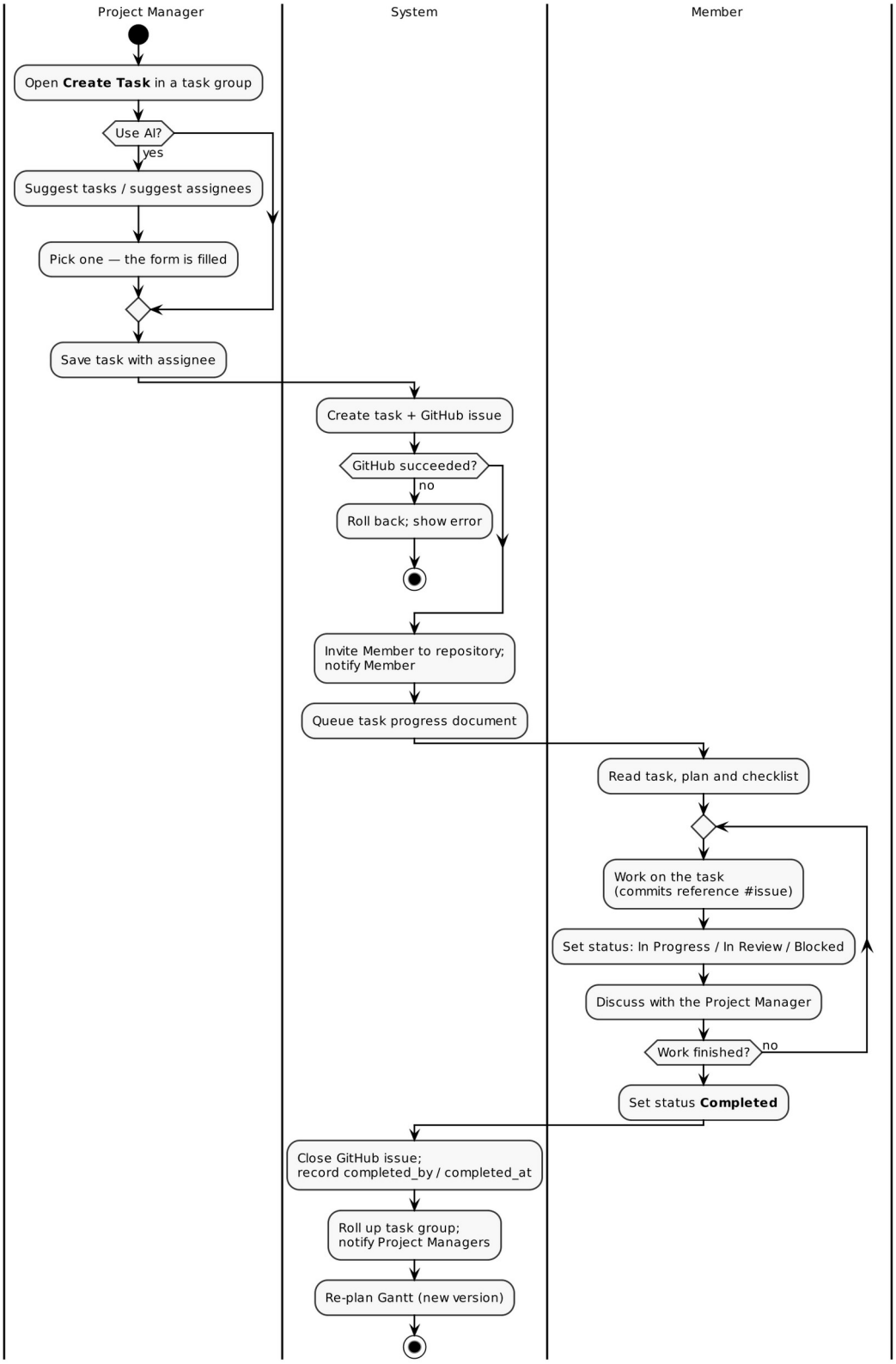


Figure 17: Activity diagram — task life cycle from creation to completion

3. SYSTEM DESIGN

System design turns the requirements of Chapter 2 into a blueprint for the software: how it is split into modules, how data is stored and kept consistent, how each process works step by step, what the user sees, and how the result will be tested.

3.1 Architecture Overview

TRACE follows a **three-tier architecture** with a separate background tier:

Table 15: Tiers of the system

Tier	Components	Role
Presentation	React single-page application (Vite build) in the browser	Three role panels, landing page, live updates over one WebSocket
Application	FastAPI on Uvicorn (ASGI)	REST API under `/api/v1`, authentication, two-layer authorisation, validation, GitHub calls, WebSocket relay
Background	Celery worker and Celery beat on Redis	AI jobs, Gantt re-planning, scheduled sweeps and reports, e-mail
Data	PostgreSQL, MinIO, GitHub repository files	Relational data, uploaded source files, generated Markdown documents
External	GitHub REST API, OpenAI-compatible model, SMTP server	Issues and repositories, AI, e-mail

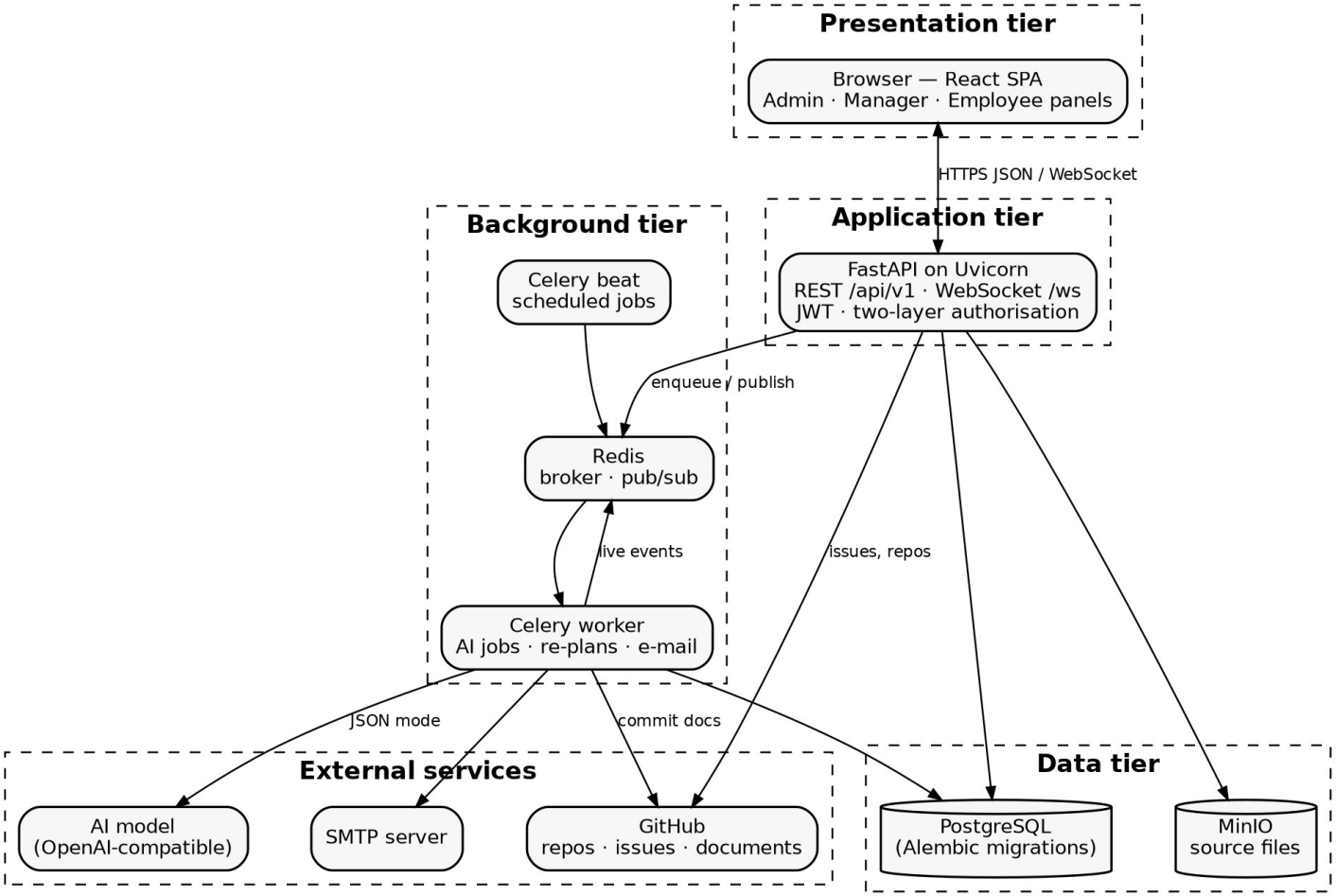


Figure 18: System architecture of TRACE

3.2 Modularization Details

The system is divided into the following modules. Each backend module is a FastAPI router (the API surface) backed by one or more service modules (the rules); the frontend mirrors them with one API client module per router and one page or component per screen.

Table 16: Modules of TRACE

No.	Module	Responsibility
M1	Authentication and Users	Login, token refresh, registration (Employee only for the public), password hashing, profile and GitHub username
M2	Access Control	Platform-role guards on every route; project-role resolution (`ProjectAccess`) for every project and task operation; status authority
M3	Project Management	Projects and their GitHub repositories, source-file upload and text extraction, Project Manager roster, candidates, project deletion with repository archiving
M4	Task Management	Task groups, tasks as GitHub issues, assignment with repository invitation, status changes with issue open/close, task-group roll-up
M5	AI Pipeline	The AI service layer (schema-validated calls with retry), Evaluate Project, people and task suggestions, task progress documents, project-document revision, weekly progress report
M6	Planning and Visualisation	Gantt scheduling rule, chart data builder and validator, version history, debounced re-evaluation, project tree graph
M7	Notifications and Discussion	Notification rows, live events through Redis publish/subscribe and the WebSocket, e-mail, per-task discussion threads
M8	Reporting	AI progress and report documents; CSV report of the project tree for a date range; performance scores used by AI ranking
M9	Administration	Control Panel (generic CRUD), Workforce with bulk upload, Configuration (GitHub, mail) with encrypted secrets, Monitor (service health, recent jobs)
M10	Scheduler	Celery beat: nightly overdue sweep, daily due reminders, weekly progress reports, weekly performance scores

Module descriptions

M1 — Authentication and Users

The user signs in with e-mail and password. The password is compared against its bcrypt hash; on success the API returns a short-lived access token and a longer-lived refresh token, both signed JWTs, together with the user's roles. Every later request carries the access token in the `Authorization` header. Anyone may register publicly, but a public registration always creates an **Employee**; only an Admin can create Admin

or Manager accounts. Users edit their own profile, including their GitHub username, which TRACE needs to invite them to repositories and assign issues to them.

M2 — Access Control

Authorisation happens in **two layers**. The first layer is the **platform role** on the route: every route declares a guard (`require_admin`, `require_roles(...)` or `require_any_role`), and an automated test fails the build if any route lacks one. The second layer is the **project role**: every project and task operation resolves a `ProjectAccess` object, which answers *who is the caller in this project?* — Admin (reads everything, changes nothing), owning Manager, Project Manager, Member or nobody (403). The same object decides status authority and discussion visibility, and the API responses carry `can_edit`, `can_discuss` and `allowed_statuses` derived from it, so the interface never has to guess.

M3 — Project Management

A Manager creates a project; TRACE derives a repository name (slug) from the project name and creates a public repository under the company's GitHub Organization **before** saving the project, so that no project exists without its repository. Source files are uploaded to MinIO and their text is extracted immediately (PDF, DOCX, Markdown, text) so the AI never has to parse them again. The Manager appoints Project Managers from the active Employees; each appointee is invited to the repository with write access. A project can be deleted only when it has no open tasks, and its repository is archived first.

M4 — Task Management

A Project Manager creates task groups and tasks. **Task creation is issue creation**: the task row and the GitHub issue are created in one request, and if GitHub refuses, the task is rolled back. Assigning a task makes the assignee a Member of the project, invites them to the repository and assigns the issue to them (or, while their invitation is still pending, leaves the issue unassigned with a warning). Status changes go through a single endpoint that enforces status authority and keeps the issue in step: completing or cancelling closes it, re-opening re-opens it. After every change the task group's completion percentage and status are recomputed.

M5 — AI Pipeline

Every AI call goes through one function, `complete(ai, Schema, system, user, context)`, which sends the prompt in JSON mode, validates the reply against a Pydantic schema, and on failure hands the validation errors back to the model and asks again (up to two retries). The jobs built on it are: **Evaluate Project** (project document, task groups with dependencies, Gantt version 1), **suggestions** (Project Managers,

assignees, tasks — returned as the job result and never stored), **task progress document** (steps and acceptance checklist, committed to the repository), **project-document revision** (revised durations and new groups when the living document changes) and the **weekly progress report**. Every job that writes something records its failure where users will see it.

M6 — Planning and Visualisation

The Gantt chart is **never edited**; it is regenerated as a new, validated version. A deterministic scheduling rule lays the task groups out by sequence and dependencies, keeps finished groups at their actual end date, and stretches unfinished groups whose end has passed up to today so that a slip pushes everything that depends on it. Triggers (completion, overdue sweep, document change, manual button) call one function that enqueues the re-plan with a 30-second delay; a job that finds a newer version already exists does nothing, so a burst of triggers produces one version. The project tree graph is a live read of Project → Task Groups → Tasks → assignees.

M7 — Notifications and Discussion

Notifications are written through one function that never notifies the person who acted. After the database commit, a live event is published on a Redis channel and every API process relays it to that user's open WebSocket; the event only says "refetch", so the browser reloads the data through the normal authorised REST routes. E-mails are queued as background jobs. Each task has a discussion thread visible to its assignee, its assigner, the project's Project Managers and the owning Manager.

M8 — Reporting

Every Monday the AI reads each task's progress document and the current task statuses of every active project and rewrites the project's **progress document** and **report document** in its repository. Managers and Employees can also download a **CSV report** of the project → task group → task tree for any date range. Weekly performance scores (completed ÷ assigned tasks due that week, and on-time rate) are computed for use as evidence in AI ranking.

M9 — Administration

The Admin has a **Control Panel** with generic create/read/update/delete over every table except the secret settings table; **Workforce**, a curated interface over employees, managers, leaves, holidays, attendance, shifts, departments and designations with CSV bulk upload that reports rejected rows; **Configuration** for the GitHub Organization, token and committer identity and for the mail server, each with a connection test and write-only secrets; and **Monitor**, showing the health of every service and the outcome of recent background jobs.

M10 — Scheduler

Celery beat runs four scheduled jobs: the overdue sweep every night at 23:00, due-tomorrow reminders every day at 08:00, the progress-report roll-up every Monday at 07:00 and the performance scores every Sunday at 23:30.

Source code organisation

Table 17: Backend package layout

Path	Contents
`app/main.py`	Application factory, middleware, router registration, WebSocket endpoint
`app/config.py`	Settings loaded from the single `.env` file; production pre-flight checks
`app/models/`	SQLAlchemy ORM classes (users, employees, projects, tasks, workforce)
`app/schemas/`	Pydantic request/response schemas, AI output schemas, Gantt chart schema
`app/api/v1/`	Routers: auth, users, employees, projects, tasks, notifications, settings, workforce, admin, services, attendance, hrms
`app/core/`	Security (JWT, bcrypt), platform-role guards, project access
`app/services/`	Business rules: project, GitHub, AI, AI jobs, Gantt, notifications, realtime, storage, extraction, scoring, reports, settings
`app/tasks.py`, `app/celery_app.py`	Celery task wrappers and the beat schedule
`alembic/versions/`	Database migrations — the only thing that creates tables
`tests/`	Automated tests with fake GitHub, fake AI and fake storage

Table 18: Frontend layout

Path	Contents
`src/App.jsx`	Route table with guest-only and role guards
`src/layouts/`	AdminLayout (sidebar), ManagerLayout and EmployeeLayout (top bar with project selector)
`src/pages/`	Landing, login, 403; admin, manager and employee pages
`src/components/project/`	Project overview, evaluation card, task board, team panel, suggestions, progress report, document editor, task discussion
`src/components/`	Gantt chart, project tree graph, notification bell, service monitor, Markdown renderer
`src/contexts/`	Authentication, theme, live-event (WebSocket) contexts
`src/api/`	One API client module per backend router
`src/lib/`	Shared helpers — error messages, dates, status colours, job polling

3.3 Data Integrity and Constraints

Data integrity means the data stays correct and consistent however the system is used. TRACE enforces it at three levels — the database, the API schemas and the business rules.

Database-level constraints

- **Entity integrity** — every table has a surrogate primary key `id BIGSERIAL`.
- **Referential integrity** — every relationship is a `FOREIGN KEY`; a task cannot point to a project, group or user that does not exist.
- **Uniqueness** — `users.email`, `users.username`, `roles.role_name`, `employees.employee_code`; the pairs (`user_id`, `role_id`) in `user_roles`, (`project_id`, `user_id`) in `project_assignments` and (`category`, `key`) in `system_settings`.
- **NOT NULL** — every mandatory column (names, titles, statuses, foreign keys that must exist, `created_at`).
- **Defaults** — `status` columns default to their starting state (`Draft`, `Pending`, `Active`), `completion_percentage` to 0, `is_read` to false, `created_at` to the current time.
- **Soft delete** — rows are marked with `deleted_at` instead of being removed, so history and references survive; every query filters out deleted rows.

Domain constraints (API level)

Every request body is validated by a Pydantic schema before any code runs:

- statuses are restricted to fixed sets — task: *Pending, In Progress, In Review, Completed, Blocked, Overdue, Cancelled*; editable project status: *Draft, Active, Completed, Archived*; priority: *Low, Medium, High, Critical*;
- lengths and ranges — project and group names 2–255 / 1–255 characters, task titles 2–255, estimated hours 0–999, group durations at least 1 day, discussion messages 1–5000 characters;
- uploads — only PDF, DOCX, MD and TXT files, at most 20 MB each;
- every AI response is validated against its schema, including rules that need data (for example, a suggested user id must be one of the candidates, a task group may only depend on an *earlier* group, 1–20 groups with unique names and 1–365 days each);
- every Gantt version is validated against the `GanttChartData` schema (dates in range, bars ordered, dependencies between existing bars, completion 0–100).

Business-rule constraints

- **Status authority** — only the assignee (Pending, In Progress, In Review, Blocked, Completed) or the Project Manager (any status except Overdue) may change a task's status; Overdue is set only by the system.
- **All-or-nothing records** — a project is not saved without its repository; a task is not saved without its GitHub issue.
- **Write-once document** — the initial project document is never rewritten after evaluation; only the living project document is edited.
- **Append-only Gantt** — Gantt versions are only ever added, never updated.
- **Safe deletion** — a task group holding tasks cannot be deleted; a project with open tasks cannot be deleted.
- **Evaluation guard** — evaluation is refused while one is running, without extracted text, or once tasks exist.
- **Transactions** — each request runs in one database transaction; failures roll back explicitly.

3.4 Database Design

The database is PostgreSQL. It contains the main tables described below and a set of supporting (auxiliary) tables that are not detailed here: `profiles`, `employee_details`, `departments`, `designations`, `work_schedules`, `leaves`, `holidays`, `attendance`, `time_zones`, `task_types`, `task_details`, `task_comments` and `task_time_logs`.

Common columns. Every table inherits four columns from a common base class: `id BIGSERIAL PRIMARY KEY`, `created_at TIMESTAMPTZ NOT NULL DEFAULT now()`, `updated_at TIMESTAMPTZ` and `deleted_at TIMESTAMPTZ` (soft delete). They are listed once in each table below for completeness. Defaults such as `status = 'Draft'` or `completion_percentage = 0` are applied by the application (ORM) when a row is inserted; `created_at` is a database default.

Table 19: List of main tables

No.	Table	Purpose
1	users	Every person who can log in
2	roles	The three platform roles
3	user_roles	Which user holds which role
4	employees	Workforce record linking a user to the organisation
5	projects	The core project entity
6	project_source_files	Uploaded requirement documents and their extracted text
7	project_assignments	Who is on a project and in which project role

No.	Table	Purpose
8	task_groups	Modules of work — one Gantt bar each
9	tasks	Units of work — one GitHub issue each
10	gantt_charts	Versioned Gantt chart snapshots
11	task_discussions	Per-task discussion messages
12	notifications	In-app notifications
13	system_settings	Runtime configuration, secrets encrypted
14	productivity_scores	Weekly task-based performance scores

Table 20: Data dictionary — users

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique user id
name	VARCHAR(150)	NOT NULL	Full name
username	VARCHAR(100)	UNIQUE, NOT NULL	Login name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login e-mail
password	TEXT	NOT NULL	bcrypt hash — never plain text
phone	VARCHAR(20)		Phone number
gender	VARCHAR(20)		Gender
dob	DATE		Date of birth
profile_id	BIGINT	FK → profiles.id	Extended profile
is_verified	BOOLEAN	NOT NULL, DEFAULT FALSE	E-mail verified
status	VARCHAR(20)	NOT NULL, DEFAULT 'Active'	Active, Inactive, Blocked, Password Changed
github_username	VARCHAR(100)		Used for repository invitations and issue assignment
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation time
updated_at	TIMESTAMPTZ		Last update
deleted_at	TIMESTAMPTZ		Soft-delete time

Table 21: Data dictionary — roles

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Role id
role_name	VARCHAR(50)	UNIQUE, NOT NULL	Admin, Manager or Employee
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 22: Data dictionary — user_roles

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
user_id	BIGINT	FK → users.id, NOT NULL	The user
role_id	BIGINT	FK → roles.id, NOT NULL	The role held
—	—	UNIQUE (user_id, role_id)	A role is held once
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 23: Data dictionary — employees

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Employee id
user_id	BIGINT	FK → users.id, NOT NULL	Login account
employee_code	VARCHAR(50)	UNIQUE, NOT NULL	Organisation's employee code
manager_id	BIGINT	FK → employees.id	Reporting manager (self reference)
department_id	BIGINT	FK → departments.id	Department
designation_id	BIGINT	FK → designations.id	Designation
work_schedule_id	BIGINT	FK → work_schedules.id	Shift
employee_details_id	BIGINT	FK → employee_details.id	Personal details
joining_date	DATE		Date of joining
status	VARCHAR(20)	NOT NULL, DEFAULT 'Active'	Active, Inactive, Blocked
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 24: Data dictionary — projects

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Project id
name	VARCHAR(255)	NOT NULL	Project name
description	TEXT		Short description
created_by_user_id	BIGINT	FK → users.id, NOT NULL	Owning Manager

Column	Data Type	Constraints	Description
github_repo	VARCHAR(255)		`organization/repository`, created by TRACE
github_default_branch	VARCHAR(100)	NOT NULL, DEFAULT 'main'	Branch for documents
status	VARCHAR(20)	NOT NULL, DEFAULT 'Draft'	Draft, Evaluating, Active, Completed, Archived
start_date	DATE		Planned start
end_date	DATE		Planned end
project_initial_doc	VARCHAR(500)		URL — AI project document, written once
project_doc	VARCHAR(500)		URL — living project document
project_progress_doc	VARCHAR(500)		URL — weekly AI progress document
project_report_doc	VARCHAR(500)		URL — weekly AI report
last_evaluated_at	TIMESTAMPTZ		Last successful evaluation
ai_errors	JSONB		`{job: {message, at}}` — last failure of each AI job
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 25: Data dictionary — project_source_files

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	File id
project_id	BIGINT	FK → projects.id, NOT NULL	Owning project
file_name	VARCHAR(255)	NOT NULL	Original file name
storage_key	VARCHAR(500)	NOT NULL	Object key in MinIO
content_type	VARCHAR(100)		MIME type
size_bytes	BIGINT		File size
extracted_text	TEXT		Text extracted at upload, read by the AI
uploaded_by_user_id	BIGINT	FK → users.id	Uploader
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 26: Data dictionary — project_assignments

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id

Column	Data Type	Constraints	Description
project_id	BIGINT	FK → projects.id, NOT NULL	Project
user_id	BIGINT	FK → users.id, NOT NULL	Employee
project_role	VARCHAR(30)	NOT NULL	Project Manager or Member
assigned_by_user_id	BIGINT	FK → users.id	Who appointed / assigned
—	—	UNIQUE (project_id, user_id)	One role per person per project
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 27: Data dictionary — task_groups

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Group id
project_id	BIGINT	FK → projects.id, NOT NULL	Project
name	VARCHAR(255)	NOT NULL	Module name
description	TEXT		What the module delivers
sequence	INTEGER	NOT NULL, DEFAULT 0	Order in the plan
estimated_duration_days	INTEGER		AI estimate
planned_start_date	DATE		Gantt bar start (current forecast)
planned_end_date	DATE		Gantt bar end (current forecast)
actual_start_date	DATE		Stamped when work starts
actual_end_date	DATE		Stamped when completed
completion_percentage	NUMERIC(5,2)	NOT NULL, DEFAULT 0	Rolled up from tasks
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Pending, In Progress, Completed, Blocked
created_by	VARCHAR(20)	NOT NULL, DEFAULT 'Human'	AI or Human
depends_on	JSONB		Ids of the groups this one waits for
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 28: Data dictionary — tasks

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Task id

Column	Data Type	Constraints	Description
project_id	BIGINT	FK → projects.id	Project
task_group_id	BIGINT	FK → task_groups.id	Task group
title	VARCHAR(255)	NOT NULL	Title (also the issue title)
description	TEXT		Details
type_id	BIGINT	FK → task_types.id	Task type
priority	VARCHAR(20)	NOT NULL, DEFAULT 'Medium'	Low, Medium, High, Critical
impact_type	VARCHAR(50)		Impact category
estimated_hours	NUMERIC(5,2)		Estimate
due_date	DATE		Deadline
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Pending, In Progress, In Review, Completed, Blocked, Overdue, Cancelled
assigned_to_user_id	BIGINT	FK → users.id	Assignee (a Member)
assigned_by_user_id	BIGINT	FK → users.id	Assigning Project Manager
completed_by	VARCHAR(20)		Assignee or Project Manager
completed_at	TIMESTAMPTZ		Completion time
github_issue_id	VARCHAR(100)		Issue number
github_issue_url	VARCHAR(255)		Issue link
task_progress_doc	VARCHAR(500)		URL — AI step plan and checklist
progress_doc_error	TEXT		Why the AI could not write it
health_score	VARCHAR(20)	NOT NULL, DEFAULT 'Green'	Green, Yellow, Red
health_reason	TEXT		Explanation of the health score
last_health_check	TIMESTAMPTZ		Last health computation
commit_hash, commit_verified, llm_verification_notes	VARCHAR(100), BOOLEAN, TEXT	commit_verified NOT NULL DEFAULT FALSE	Retained legacy values, no longer written
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 29: Data dictionary — gantt_charts

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
project_id	BIGINT	FK → projects.id, NOT NULL	Project
version	INTEGER	NOT NULL, DEFAULT 1	Incremented per regeneration

Column	Data Type	Constraints	Description
chart_data	JSONB		Validated render payload: range, bars, tasks, dependencies, milestones
generated_reason	VARCHAR(100)		initial, task_status_change, project_doc_updated, manual
generated_at	TIMESTAMPTZ		Generation time
generated_by_job	VARCHAR(64)		Background job id that wrote it
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 30: Data dictionary — task_discussions

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Message id
task_id	BIGINT	FK → tasks.id, NOT NULL	Task discussed
user_id	BIGINT	FK → users.id, NOT NULL	Author
parent_id	BIGINT	FK → task_discussions.id	Thread root (self reference)
message	TEXT	NOT NULL	Message text
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 31: Data dictionary — notifications

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Notification id
user_id	BIGINT	FK → users.id, NOT NULL	Recipient
type	VARCHAR(50)	NOT NULL	task_assigned, task_due, task_completed, discussion_reply, project_evaluated
title	VARCHAR(255)	NOT NULL	Headline
body	TEXT		Detail
link	VARCHAR(500)		Deep link into the recipient's panel
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	Read flag
related_task_id	BIGINT	FK → tasks.id	Related task
related_project_id	BIGINT	FK → projects.id	Related project

Column	Data Type	Constraints	Description
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 32: Data dictionary — system_settings

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
category	VARCHAR(50)	NOT NULL	github or mail
key	VARCHAR(100)	NOT NULL	Setting name, e.g. token, owner, host
value	TEXT		Plain value, or a Fernet-encrypted token for secrets
is_secret	BOOLEAN	NOT NULL, DEFAULT FALSE	Secret values are never returned by any API
—	—	UNIQUE (category, key)	One value per setting
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 33: Data dictionary — productivity_scores

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
employee_id	BIGINT	FK → employees.id, NOT NULL	Employee scored
week_start	DATE	NOT NULL	Monday of the week
task_score	NUMERIC(5,2)		Completed ÷ assigned tasks due that week × 100
deadline_score	NUMERIC(5,2)		Completed on time ÷ completed × 100
final_score	NUMERIC(5,2)		Mean of the two scores
rating	VARCHAR(15)		Excellent, Good, Average, Risk
att_score, active_time_score	NUMERIC(5,2)		Retained, left empty — attendance is not a scoring input
anomaly_flags	JSONB		Flags raised for the week
generated_at	TIMESTAMPTZ		Computation time
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Relationships Table

Table 34: Relationships between tables

Parent table	Child table	Relationship	Through column
users	user_roles	One-to-Many	user_roles.user_id
roles	user_roles	One-to-Many	user_roles.role_id
users	employees	One-to-One	employees.user_id
employees	employees	One-to-Many (self join)	employees.manager_id
users	projects	One-to-Many	projects.created_by_user_id
projects	project_source_files	One-to-Many	project_id
projects	project_assignments	One-to-Many	project_id
users	project_assignments	One-to-Many	user_id
projects	task_groups	One-to-Many	project_id
task_groups	tasks	One-to-Many	task_group_id
projects	tasks	One-to-Many	project_id
users	tasks	One-to-Many	assigned_to_user_id, assigned_by_user_id
projects	gantt_charts	One-to-Many	project_id
tasks	task_discussions	One-to-Many	task_id
task_discussions	task_discussions	One-to-Many (self join)	parent_id
users	notifications	One-to-Many	user_id
employees	productivity_scores	One-to-Many	employee_id

3.5 Procedural Design (Process Logic)

The process logic of the main operations is given below as numbered steps.

Login

1. The user enters e-mail and password on the login page and presses *Sign in*.
2. The API finds the active user with that e-mail; if none exists, it answers "Invalid credentials".
3. The password is checked against the stored bcrypt hash; on mismatch, "Invalid credentials".
4. The API issues an access token and a refresh token (JWT, signed with the server secret) and returns them with the user's roles.
5. The browser stores the token and routes the user to `/admin`, `/manager` or `/employee` by role.

Create project

1. The Manager enters the name and description, adds source files and reviews the repository name shown.
2. The API resolves the GitHub Organization and token from Configuration (database value, else `.env`); if either is missing, it answers 503.
3. The name is turned into a slug (lower-case, other characters to `-`, at most 100 characters).
4. The API creates the public repository under the Organization. A name clash answers 409; any other GitHub failure answers 502 and nothing is saved.
5. The project row is saved with `status = Draft` and the repository name.
6. Each source file is stored in MinIO and its text is extracted and saved; unsupported or oversized files are reported back.

Evaluate project

1. The owner presses *Evaluate Project*. The API refuses (409) if the project is already evaluating, has no extracted text or already has tasks, and (503) if the AI or GitHub is not configured.
2. The project is set to *Evaluating*, committed, and the `evaluate_project` job is queued; the API returns the job id.
3. The job reads the extracted text of all source files (at most 60,000 characters).
4. The AI writes the project document; it is committed to `docs/project_initial_doc.md` and `docs/project_doc.md`.
5. The AI plans the task groups (name, description, duration, dependencies); the plan is validated.
6. Old groups are soft-deleted and the new groups saved; the scheduling rule assigns dates; Gantt version 1 is saved with reason *initial*.
7. The project becomes *Active*, the owner is notified; on any failure the transaction is rolled back, the project returns to its previous state and the failed step is recorded in `ai_errors`.

Create task

1. The Project Manager fills in the task (optionally using *Suggest tasks* and *Suggest assignees*) and saves.
2. The API checks the caller is a Project Manager of the project and that the group belongs to it.
3. The task row is flushed. If an assignee is chosen, they become a Member and are invited to the repository.

4. The GitHub issue is created (assigned to the assignee's GitHub account if possible). If GitHub fails, the transaction is rolled back and nothing is kept.
5. The issue number and URL are stored, the assignee is notified, the group is rolled up and the transaction committed.
6. The `generate_task_progress_doc` job is queued to write the task's plan and checklist.

Update task status

1. The user chooses a new status (the buttons shown are exactly the server's `allowed_statuses`).
2. The API checks status authority; a caller without it receives 403.
3. If the task moves between open and closed, the GitHub issue is closed (completed / not planned) or re-opened.
4. On *Completed*, `completed_by` and `completed_at` are set and the Project Managers (and the assignee, if someone else completed it) are notified; otherwise they are cleared.
5. The group is rolled up and the transaction committed; if completion changed, a Gantt re-plan is requested.

Gantt re-plan

1. A trigger calls `request_reevaluation(project, reason)`, which queues the job with a 30-second delay (none for the manual button).
2. When it runs, the job exits if a version newer than its request time already exists.
3. For *project_doc_updated*, the AI proposes revised durations for unfinished groups and new groups for uncovered features.
4. The scheduling rule re-computes every group's planned dates.
5. A new chart is built, validated and saved as the next version with its reason and job id.

Weekly progress report

1. Every Monday at 07:00, one job is queued per active project.
2. The job reads every task's progress document from the repository and the current task statuses.
3. The AI writes the progress document and the report document.
4. Both are committed to the repository and their URLs stored on the project.

Object-Oriented Design

The backend is object-oriented. The main classes and their responsibilities are:

Table 35: Main classes and responsibilities

Class	Responsibility
`BaseModelMixin`	Common id, timestamps and soft-delete columns for every entity
`User`, `Role`, `UserRole`	Identity and platform roles
`Employee`	Workforce record of a user
`Project`, `ProjectSourceFile`, `ProjectAssignment`	Project, its documents and its team
`TaskGroup`, `Task`	Modules and units of work
`GanttChart`	One immutable version of a project's chart
`TaskDiscussion`, `Notification`	Communication records
`SystemSetting`	A configuration value, encrypted when secret
`ProjectAccess`	The caller's rights in one project: owner, Project Manager, assignee; `may_set_status`, `allowed_statuses`, `may_discuss`
`GitHubClient`	Every GitHub call: repositories, collaborators, issues, files
`AIClient`	The model connection in JSON mode
`GanttChartData`	Schema and validator of a Gantt version
`Outbox`	Notifications collected in a transaction, delivered after the commit
`Settings`	Configuration from `.env`, with production checks

3.6 User Interface Design

The user interface was designed around the following principles:

- **One audience per panel.** Admin, Manager and Employee each have their own layout and base route; nothing a role cannot use is shown to it.
- **The project is the unit of navigation.** Manager and Employee panels have a project selector in the top bar, defaulting to "All" (a combined dashboard); the URL always identifies the selected project.
- **The server decides what the user may do.** Buttons such as status changes and *Edit* are rendered from the permissions in the API response.
- **Long work never blocks the screen.** AI actions show a running state, poll for the result and report failures with their reason.
- **Consistency.** One component library, design tokens for every colour, one icon set, the same page shell everywhere, and light and dark themes.

- **Feedback.** Every action ends in a toast message; lists and AI surfaces have empty and error states.

Table 36: Screens of the system

Screen	Panel	Purpose
Landing page	Public	Product overview, how it works, features, sign-in link
Login	Public	E-mail and password sign-in
403 page	All	Shown when a user opens another role's panel
Control Panel	Admin	Table picker and generic data grid with create, edit, delete
Workforce	Admin	Tabs for employees, managers, leaves, holidays, attendance, shifts, departments, designations; bulk upload
Monitor	Admin	Health cards for every service; recent job outcomes
Configuration	Admin	GitHub and mail settings with connection tests
Manager dashboard	Manager	Project counts, progress, at-risk projects, activity, report download, *New Project*
Create project wizard	Manager	Details → source files → review → create
Project page	Manager	Overview (evaluation card, documents, files), Tree, Gantt, Progress Report, Team
Employee dashboard	Employee	Progress across projects, my tasks by due state, overdue callout, report download
Project page (PM)	Employee	Overview, Tree, Gantt, Tasks (board with *Create Task*), Project Doc editor, Team
Project page (Member)	Employee	Overview, Tree, Gantt, My Tasks
Task detail	Employee / Manager	Description, GitHub issue, progress document with checklist, status buttons, discussion
Notification bell	All	Unread count, latest notifications, deep links, mark all read
Settings	All	Own profile, GitHub username, password

Screenshots of the running application are given in the appendix *Input and Output Screens*.

3.7 Test Case Design

Test cases were designed before and during implementation, at two levels.

Unit test cases exercise one function or endpoint in isolation, with the external services (GitHub, the AI model, MinIO, Redis) replaced by fakes that behave like the real

services. They cover each module's rules — validation, authorisation, the task–issue rule, the scheduling rule, schema validation of AI output.

System test cases exercise complete user flows through the API across modules — for example, *a Manager creates a project, appoints a Project Manager, who creates a task that becomes a GitHub issue,* or *an evaluation writes documents, task groups and Gantt version 1*.

Techniques used in designing them:

- **Equivalence partitioning** — valid and invalid classes of input (supported and unsupported file types; statuses the caller may and may not set).
- **Boundary value analysis** — limits such as 20 MB uploads, 1–20 task groups, 1–365 days, title lengths, the 60,000-character source limit.
- **Decision tables** — who may do what: every role (owner Manager, other Manager, Admin, Project Manager, Member, outsider, anonymous) against every project route.
- **State transition testing** — task and project status changes, including re-opening.
- **Error guessing** — GitHub refusing an invitation, a pending invitee, an AI that keeps returning invalid JSON, a changed encryption key.

The designed test cases and their results are reported in Chapter 5.

4. STANDARDIZATION OF THE CODING

A fixed set of coding standards was followed throughout TRACE, so that every module looks and behaves the same way and a new developer can predict where a rule lives. The standards are written down in the project's conventions document and checked by automated tests where possible.

4.1 Naming and Layout Conventions

Table 37: Coding conventions

Item	Convention
Python names	`snake_case` for functions and variables, `PascalCase` for classes, `UPPER_CASE` for constants (PEP 8)
JavaScript names	`camelCase` for functions and variables, `PascalCase` for React components, one component per file
Database	Table names plural and `snake_case`; foreign keys named `<entity>_id`; every table has `id`, `created_at`, `updated_at`, `deleted_at`
API	Versioned under `/api/v1`; nouns for resources (`/projects/{id}/tasks`), a verb only for actions (`/evaluate`, `/reevaluate`)
Files	One router per resource in `api/v1/`, business rules in `services/`, schemas in `schemas/`, models in `models/`
Frontend styling	Only the shared component library and design tokens; no hard-coded colours; one icon library
Configuration	One environment file (`.env`) for the whole stack; nothing duplicated
Schema changes	Only through Alembic migrations, one concern per migration

4.2 Code Efficiency

- **Asynchronous I/O throughout the backend.** FastAPI routes, SQLAlchemy sessions (asyncpg driver) and the HTTP client for GitHub are all asynchronous, so one server process serves many requests while others wait on the database or on GitHub.
- **Slow work leaves the request.** Every AI call, every Gantt re-plan, every e-mail and every scheduled sweep runs in a Celery worker; the API only queues the job and returns its id. A request never waits for the AI model.
- **Debouncing.** A burst of task completions produces one Gantt re-plan, not one per completion: each trigger is delayed 30 seconds, and a job that finds a newer version already exists does nothing.

- **Deterministic where possible, AI only where needed.** Rescheduling after a status change is plain date arithmetic; the AI is consulted only when the scope changes (the project document is edited). This saves cost and removes a failure mode.
- **Extract once, read many times.** Text is extracted from uploaded files once, at upload, and stored; the AI pipeline never re-parses PDFs or Word files.
- **Bounded AI inputs.** Source text sent to the AI is capped at 60,000 characters, task documents at 4,000 characters each and candidate lists at 25 people, keeping calls fast and cheap.
- **Eager loading.** Related rows (roles of a user, the project of a task) are loaded in the same query with `selectinload`, avoiding the "N + 1 queries" problem.
- **Content-free live events.** WebSocket events only say "refetch", so the server does not serialise data twice, and the socket needs no permission logic of its own.
- **Soft delete with indexed keys.** Rows are marked deleted instead of being removed, and every lookup is by primary or foreign key.
- **Production frontend build.** In production the React application is served as a minified, pre-built bundle.

4.3 Error Handling

Errors are handled consistently at every layer:

- **One exception hierarchy.** All expected errors are subclasses of `TraceException` (an HTTP exception carrying a status code and a readable message): invalid credentials (401), expired or malformed token (401), insufficient permissions (403), not found (404), already exists (409), plus specific ones such as *AI not configured* (503) and *GitHub not configured* (503). FastAPI turns them into a JSON response `{"detail": "…"}`, which the frontend shows in a toast.
- **Meaningful status codes.** 400/422 for invalid input, 401 for authentication, 403 for authorisation, 404 for missing records, 409 for conflicts (repository name clash, evaluation already running, group still holding tasks), 502 when GitHub refuses, 503 when a required service is not configured or reachable.
- **Explicit rollback.** Where an external call follows a database write (a GitHub issue after a task row), the service rolls the transaction back itself on failure, rather than relying on the caller.
- **Warnings instead of failures where appropriate.** Problems that should not block the main change — a missing GitHub username, an invitation GitHub

refused, a pending invitation — are returned as a `warnings` list and shown to the user, while the change itself succeeds.

- **Visible AI failures.** A failed AI job records its reason where people will see it: `projects.ai_errors` (per job, with time and the step that failed) or `tasks.progress_doc_error`. It never leaves stale data looking fresh.
- **Retry on invalid AI output.** An AI answer that fails validation is sent back with the errors for correction, up to two retries.
- **No secret in an error.** Configuration errors are raised in a way that never echoes the values of settings; a changed encryption key gives a clear "enter it again" message instead of a crash.
- **Frontend handling.** A central HTTP client refreshes an expired token automatically and retries the request once; a shared helper turns any error response into a readable message; every page has loading, empty and error states.
- **Logging.** Background jobs log failures with the step and the project id, so problems can be traced in the worker log and on the Admin Monitor page.

4.4 Parameters Calling and Passing

- **Dependency injection.** FastAPI injects the database session, the current user, the GitHub client and the project access object into each route as parameters (`Depends(...)`). Routes therefore never create these themselves, and tests replace them with fakes.
- **Typed parameters.** Every Python function declares the types of its parameters and return value. Path parameters (`project_id: int`), query parameters (`start: date`, `end: date`) and request bodies (Pydantic models) are converted and validated by FastAPI before the function runs.
- **Keyword-only arguments** are used for optional behaviour (for example `notify(..., task=..., exclude=actor)`, `run_evaluate_project(db, project_id, *, job_id, ai, github)`), which makes calls self-describing and prevents arguments in the wrong order.
- **Pass by object reference.** Python passes references to objects; services receive the ORM objects they need (a `Task`, a `ProjectAccess`) rather than re-loading them, and modify them within the caller's transaction. The caller commits.
- **Jobs receive ids, not objects.** Background jobs are queued by name with plain arguments (ids, strings, ISO dates), because objects cannot cross process boundaries; the job loads fresh rows in its own session.

- **Frontend props.** React components receive data and callbacks as props (for example a task board receives the project, its tasks and an `onChanged` callback) and never fetch permissions on their own.

4.5 Validation Checks

Table 38: Validation checks

Where	Checks
Login and registration	E-mail format, required fields, unique e-mail and username, minimum password length; public registration cannot choose a privileged role
Project	Name 2–255 characters; the name must yield a valid repository slug; editable status limited to Draft, Active, Completed, Archived
Source files	Type must be PDF, DOCX, MD or TXT; size at most 20 MB; file must be readable; rejected files are listed with the reason
Task group	Name 1–255 characters; duration at least 1 day; must belong to the project
Task	Title 2–255 characters; priority one of Low, Medium, High, Critical; estimated hours 0–999; group must belong to the project; assignee must be an active Employee
Task status	One of the seven statuses; the caller must be allowed to set it (assignee or Project Manager); Overdue is refused for everyone
Project document	1–200,000 characters; an unchanged save commits nothing
Discussion	Message 1–5,000 characters; a reply must belong to the same task; caller must be allowed to discuss
Report download	Start and end dates required; end not before start
Settings	Organization name and e-mail formats; mail port range; secrets write-only
Bulk upload	UTF-8 CSV; known column names; each value coerced to its type (integer, date, time, e-mail, one of a list); required fields present; references (department, employee code) must exist
AI output	Must match its schema: lengths, ranges, unique group names, dependencies only on earlier groups, suggested people only from the candidate list
Gantt version	Range covers every bar and milestone; one bar per group; start ≤ end; completion 0–100; dependencies between existing bars and never to itself
Frontend forms	Required fields and formats are checked before submission; the server repeats every check

5. TESTING

Testing is the process of executing the software with the intent of finding errors, and of showing that it meets its requirements. TRACE was tested continuously: every build increment added its tests, and the complete suite was run before an increment was considered finished.

5.1 Testing Techniques and Testing Strategies Used

5.1.1 Testing Techniques

Black-box testing

Black-box testing examines the functionality of the system without looking at its internal structure; test cases are derived from the specification. In TRACE almost every test calls the real HTTP API as a client would and checks only the response and the visible effects (the status code, the returned data, the GitHub issue created, the notification sent). Techniques used:

- **Equivalence partitioning** — for example, file uploads were tested with one representative of each class: supported and readable, supported but unreadable, unsupported type, over the size limit.
- **Boundary value analysis** — the 20 MB upload limit, 1 and 20 task groups, 1 and 365 days, 2-character titles, the end date equal to and before the start date of a report.
- **Decision-table testing** — the *role matrix*: every combination of caller (owner Manager, other Manager, Admin, Project Manager, Member, outsider, anonymous) and project operation, with the expected answer (allowed, 403, 401).
- **State-transition testing** — task status changes in both directions (completing and re-opening, cancelling), project states through evaluation success and failure.
- **Use-case testing** — end-to-end flows such as *Manager → Project Manager → task → GitHub issue*.
- **Error guessing** — GitHub refusing an invitation, an assignee whose invitation is still pending, a repository that is already gone, an AI that keeps returning invalid JSON, a changed encryption key.

White-box testing

White-box (structural) testing uses knowledge of the internal code. It was used for the pure functions where every branch matters:

- the **scheduling rule** — groups with and without dependencies, with a hand-planned span, completed groups, and unfinished groups whose end has passed (the slip branch);
- the **AI retry loop** — success on the first attempt, success after a retry, failure after the last retry;
- the **status-authority function** — every branch of `may_set_status` for assignee, Project Manager and others;
- the **Gantt schema validator** — each rule that can reject a chart;
- the **repository slug** function — letters, digits, punctuation, long names, names without letters or digits.

Coverage criteria applied were **statement coverage** and **branch (decision) coverage** of these functions.

Grey-box testing

Many tests combine both views: they drive the system through its public API (black box) but then inspect the database or the fake GitHub's recorded state (white box) — for example, checking that after a failed evaluation *no* task group from the failed run remains in the database.

5.1.2 Testing Strategies

Table 39: Levels of testing applied

Level	How it was applied in TRACE
Unit testing	Individual functions and endpoints in isolation, with GitHub, the AI model, MinIO and Redis replaced by fakes that behave like the real services
Integration testing	Routes, services, the ORM and a real PostgreSQL test database working together; the real GitHub client code runs against a fake GitHub transport
System testing	Complete flows across modules through the HTTP API: project creation to issue, evaluation to Gantt, completion to notification and re-plan
Regression testing	The full suite was run after every change; bugs that were found received a test that pins the fix
Security testing	A static audit that every route declares an authorisation guard; role matrices; checks that secrets are never returned and are encrypted at rest; production pre-flight refusals
Migration testing	An empty database is migrated to the latest revision and compared with the models (no drift); only one migration head may exist
Live acceptance testing	An opt-in test that runs the real code path against real GitHub (create repository, name clash, commit documents, assign and close issues, archive)
User acceptance testing	Walk-through of each panel's flows by the users of each role

5.2 Testing Plan Used

Table 40: Test plan

Item	Plan
Objective	Verify every functional requirement (FR-01 … FR-30) and the security rules before an increment is accepted
Scope	All backend routes, services, background job bodies, schemas and migrations; frontend verified by building and by walk-through
Test environment	Linux; PostgreSQL and Redis in Docker; a dedicated test database `trace_test` built by Alembic migrations; Python virtual environment
Tools	pytest, pytest-asyncio, HTTPX asynchronous test client, a fake GitHub on `httpx.MockTransport`, a scripted fake AI, a fake object store
Isolation	Autouse fixtures block real GitHub and MinIO in every test, record every queued job and every live event, and pin an empty AI key, so no test reaches a real model, queue or worker
Test data	A shared "world" fixture per test: an Admin, two Managers, a Project Manager, Members and an outsider, with a project and repository
Entry criteria	Code of the increment complete; database migrated to head
Exit criteria	Whole suite passes; frontend build passes; the increment's "done when" condition demonstrated
Responsibility	The developer writes the tests together with the code of each increment
Schedule	Continuous during every build phase, followed by the dedicated testing activity (J) of the project plan

5.3 Test Reports

The suite contains **218 test functions**; the role-matrix and other parametrised tests expand them to several hundred test cases. At the Phase 5 hand-off the full run reported **374 passed and 2 skipped** (the two skipped tests are the opt-in live GitHub tests), and the notification, discussion and administration tests were added after that. The tables below report representative cases from the suite.

5.3.1 Unit Test Cases

Authentication

Table 41: Unit tests — authentication

Test Id	Test case	Expected output	Actual output	Result
UT-01	Log in with a valid e-mail and password	200; access and refresh tokens and roles returned	Tokens returned	Pass

Test Id	Test case	Expected output	Actual output	Result
UT-02	Log in with a wrong password	401 "Invalid email or password"	401 returned	Pass
UT-03	Log in with an e-mail that does not exist	401, same message (no account enumeration)	401 returned	Pass
UT-04	Log in with an empty body	422 validation error	422 returned	Pass
UT-05	Refresh with a valid refresh token	New access token	Token issued	Pass
UT-06	Refresh with an invalid token	401	401 returned	Pass
UT-07	Register without a role	Account created as Employee	Employee created	Pass
UT-08	Anonymous registration asking for Admin	403 — privileged roles need an Admin	403 returned	Pass
UT-09	Admin registers a Manager	201	Manager created	Pass
UT-10	Register with an e-mail already used	409 already exists	409 returned	Pass

Projects and GitHub

Table 42: Unit tests — projects and repositories

Test Id	Test case	Expected output	Actual output	Result
UT-11	Admin or Project Manager tries to create a project	403	403 returned	Pass
UT-12	Manager creates "Mobile App: v2 (Beta)!"	Public repo `acme/mobile-app-v2-beta` created with issues enabled; project Draft	As expected	Pass
UT-13	Project name with no letters or digits	422 — no valid repository name	422 returned	Pass
UT-14	Repository name already exists	409 naming the repository; nothing created	409, no row	Pass
UT-15	No GitHub token or organization configured	503; no project	503 returned	Pass
UT-16	Token lacks the Administration permission	502 with a precise message	Message returned	Pass
UT-17	Upload PDF, DOCX, MD and an unsupported file	Supported files stored and text extracted; unsupported reported	As expected	Pass
UT-18	A non-owner uploads a source file	403	403 returned	Pass
UT-19	Delete a project with open tasks	409; project kept	409 returned	Pass

Test Id	Test case	Expected output	Actual output	Result
UT-20	Delete a project; archiving the repo fails	Project kept, error returned	As expected	Pass
UT-21	Managers list projects	Each sees only their own	As expected	Pass

Team and assignment

Table 43: Unit tests — team and assignment

Test Id	Test case	Expected output	Actual output	Result
UT-22	Appoint a Project Manager	PM row created; invited to the repo with Write	As expected	Pass
UT-23	Appoint a PM with no GitHub username	Appointed, with a warning	Warning returned	Pass
UT-24	GitHub refuses the invitation	Appointment still succeeds, with a warning	As expected	Pass
UT-25	Appoint a Manager or Admin as PM	422 — only Employees	422 returned	Pass
UT-26	Assign a task to a pending invitee	Issue left unassigned, "invitation pending" warning	As expected	Pass
UT-27	Remove a PM who holds no tasks	Access revoked, pending invitation withdrawn	As expected	Pass
UT-28	Remove a PM who holds tasks	Stays as Member, keeps repository access	As expected	Pass

Tasks and status authority

Table 44: Unit tests — tasks and status authority

Test Id	Test case	Expected output	Actual output	Result
UT-29	Member tries to create a task	403	403 returned	Pass
UT-30	Create a task; GitHub fails	Task rolled back; no row	No row	Pass
UT-31	Create a task in another project's group	422	422 returned	Pass
UT-32	Assignee sets own task to Completed	200; completed_by = Assignee; issue closed	As expected	Pass
UT-33	Assignee tries Cancelled or Overdue	403	403 returned	Pass
UT-34	Member changes someone else's task	403	403 returned	Pass
UT-35	Manager or Admin tries to complete a task	403	403 returned	Pass

Test Id	Test case	Expected output	Actual output	Result
UT-36	Status sent through the generic edit endpoint	Ignored — status unchanged	Unchanged	Pass
UT-37	PM completes a task	Issue closed; group roll-up updated	As expected	Pass
UT-38	`allowed_statuses` for each role	Exactly the statuses `may_set_status` allows	Match	Pass
UT-39	Delete a task	Soft-deleted; issue closed as not planned	As expected	Pass
UT-40	Delete a group that holds tasks	409	409 returned	Pass

AI pipeline, Gantt and scheduling

Table 45: Unit tests — AI pipeline and Gantt

Test Id	Test case	Expected output	Actual output	Result
UT-41	AI returns invalid JSON once, then valid	Errors sent back; valid result accepted	As expected	Pass
UT-42	AI returns invalid JSON three times	`AIOutputInvalid` raised; nothing saved	As expected	Pass
UT-43	Plan with repeated names or a dependency on a later group	Rejected by the schema	Rejected	Pass
UT-44	Suggestion naming a non-candidate user	Rejected by the schema	Rejected	Pass
UT-45	Evaluate while running / without text / with tasks	409 each	409 returned	Pass
UT-46	Evaluate without AI or GitHub configured	503	503 returned	Pass
UT-47	Scheduling with dependencies and a late group	Dependants start after; slip stretches to today	As expected	Pass
UT-48	Save an inconsistent Gantt chart	Rejected; nothing persisted	As expected	Pass
UT-49	Versions are appended	New version number each time; old kept	As expected	Pass
UT-50	No Gantt yet	`null`, not an error	`null`	Pass

Settings, notifications and administration

Table 46: Unit tests — settings, notifications and administration

Test Id	Test case	Expected output	Actual output	Result
UT-51	Save the GitHub token	Stored encrypted; never returned — only "set"	As expected	Pass
UT-52	Omit the token on save / send empty	Omitted keeps it; empty clears it	As expected	Pass
UT-53	Encryption key changed after saving	Clear 503 asking to re-enter the secret	As expected	Pass
UT-54	Generic Control Panel CRUD on `system_settings`	Table not available	Not listed	Pass
UT-55	Test connection with a bad token	Failed check with GitHub's message	As expected	Pass
UT-56	Mail body containing HTML	Escaped in the e-mail	Escaped	Pass
UT-57	Bulk upload with good and bad rows	Good rows added; bad rows reported with line numbers	As expected	Pass
UT-58	Notifications without a login	401	401 returned	Pass
UT-59	WebSocket with a bad token	Connection refused	Refused	Pass
UT-60	Report with end before start	422	422 returned	Pass

5.3.2 System Test Cases

Table 47: System tests

Test Id	Scenario	Steps and expected outcome	Actual outcome	Result
ST-01	Manager to issue	Manager creates project → appoints PM → PM creates group and task with assignee → GitHub issue #1 exists, titled as the task, assigned to the Member's GitHub account, body contains the description	As expected	Pass
ST-02	Evaluate Project	Upload source text → evaluate → project document committed twice, task groups created by AI, Gantt v1 with reason *initial*, project Active, owner notified	As expected	Pass
ST-03	Failed evaluation	AI fails at planning → project back to Draft, `ai_errors` names the step, no groups from the failed run remain, owner notified of failure	As expected	Pass

Test Id	Scenario	Steps and expected outcome	Actual outcome	Result
ST-04	Completion flow	Assignee completes task → issue closed, group roll-up updated, PMs notified live and by e-mail, Gantt re-plan queued with 30 s delay	As expected	Pass
ST-05	Burst of completions	Several completions in quick succession → exactly one new Gantt version	One version	Pass
ST-06	Project document edit	PM saves a changed document → AI revises durations / adds groups → new version *project_doc_updated*; unchanged save commits nothing	As expected	Pass
ST-07	AI failure on re-plan	AI call fails during a document re-plan → chart still re-planned, `ai_errors` records the failure	As expected	Pass
ST-08	Overdue sweep	Past-due open tasks → Overdue, assignee and PMs notified, one re-plan per project	As expected	Pass
ST-09	Due reminders	Only tasks due tomorrow produce a reminder	As expected	Pass
ST-10	Discussion	Assignee posts, PM replies → both see the thread; an outsider Member and the Admin get 403; participants notified	As expected	Pass
ST-11	Weekly progress report	Job reads task documents → progress and report documents committed, URLs stored	As expected	Pass
ST-12	Report download	Manager downloads CSV for a date range → only their projects, tasks created, due or completed in range	As expected	Pass
ST-13	Role matrix	Every project route × {owner, other Manager, Admin, PM, Member, outsider, anonymous} → expected allow / 403 / 401	All as expected	Pass
ST-14	Route guard audit	Every registered route declares a guard; only `/health` and login / refresh / register are public	As expected	Pass
ST-15	Fresh installation	Empty database migrated to head; no difference from the models; application start does not create tables	As expected	Pass
ST-16	Production pre-flight	Production refuses weak JWT secret, debug mode, missing encryption key and seed passwords, without printing secrets	As expected	Pass
ST-17	Live GitHub (opt-in)	Real organization: connection test, repository creation, name clash, document commit, issue assign / close, archive on delete	Passed	Pass

5.4 Debugging and Code Improvement

Debugging was driven by failing tests and by review of each increment. The most instructive defects found, and the improvements made, were:

Table 48: Defects found and improvements made

No.	Defect or weakness found	Correction / improvement
1	A legacy generic task-update endpoint accepted a `status` field, so any signed-in user could mark any task complete	Removed; status got its own endpoint that enforces status authority in one place
2	Public registration trusted a caller-supplied role, so anyone could register as Admin	Public registration now always creates an Employee; privileged accounts need an Admin token
3	Several service endpoints were reachable without authentication	Every route now declares a guard, and a static audit test fails if one does not
4	A fresh database could not reach the latest migration, because some tables had been created at start-up by `create_all` instead of by a migration	A catch-up migration was inserted and `create_all` removed; a test now migrates an empty database and checks for drift
5	Creating a task flushed the row before calling GitHub; on a GitHub error the row could survive	The service rolls back explicitly on failure; the test harness was changed to behave like the real session
6	Reading `created_at` of a new row raised an asynchronous lazy-load error	Rows are refreshed after insert before being returned
7	The attendance log endpoint crashed on columns that no longer existed	Fixed; found by the new role matrix
8	The generic Control Panel exposed the settings table, which would have returned the token and stored it unencrypted	Table removed from the generic CRUD; a dedicated settings API is the only writer
9	A malformed encryption key produced an unexplained server error	Now a clear 503 naming the key; production refuses to start with it
10	GitHub refuses a pending invitee as issue assignee	The service retries unassigned and warns; the fake GitHub enforces the same rule
11	Background jobs shared the web server's connection pool across event loops	Each job uses its own session without pooling
12	The old chat WebSocket was an open, unauthenticated broadcast room	Removed; the new socket requires a valid token and carries no data
13	E-mail bodies inserted text into HTML unescaped	Escaped

No.	Defect or weakness found	Correction / improvement
14	The production pre-flight, as first written, printed part of a secret in its error	Moved so that errors never echo settings; a test with canary values pins it
15	An AI call on every Gantt re-plan added cost and a failure mode	Re-plans after status changes became deterministic; the AI is consulted only when the project document changes

6. SYSTEM SECURITY MEASURES

Security was treated as a requirement of every increment, not a final step. This chapter describes how TRACE protects its data and how user profiles and access rights are created and enforced.

6.1 Database and Data Security

Protection of credentials and secrets

- **Password hashing.** User passwords are hashed with **bcrypt** using 12 rounds of salting. The plain password is never stored or logged, and a hash cannot be reversed; login compares the entered password against the hash.
- **Encryption of stored secrets.** Secrets entered by the Admin — the GitHub token and the SMTP password — are encrypted with **Fernet** (authenticated symmetric encryption) before they are written to `system_settings`. The encryption key lives only in the server's `.env` file, never in the database. A reader of the database, a backup or the Control Panel therefore sees only ciphertext.
- **Write-only secrets.** No API ever returns a secret. The Configuration screen shows only whether a token is set and where it comes from (*Not set / Set here / Set in .env*). The settings table is deliberately excluded from the Admin's generic table editor, which would otherwise be able to read or overwrite it unencrypted.
- **One configuration file.** Database, Redis, MinIO, AI and JWT credentials are kept in a single `.env` file on the server, excluded from version control; a template without values is provided instead.

Protection of data in the database

- **No direct access for end users.** Users reach data only through the API; the application connects to PostgreSQL with a single role whose password is known only to the server.
- **SQL injection prevention.** All queries are built with the SQLAlchemy ORM and executed with bound parameters; user input is never concatenated into SQL.
- **Input validation.** Every request body, path and query parameter is validated by Pydantic schemas before it reaches the business logic (types, lengths, ranges, allowed values).
- **Referential integrity and constraints** (Section 3.3) prevent inconsistent data even if a bug slipped past the application.
- **Soft deletion.** Records are marked deleted instead of being erased, preserving history and references; deleted rows are excluded from every query.

- **Atomic changes.** Each request runs in one transaction; a change that also needs GitHub (a task and its issue, a project and its repository) is either completed fully or rolled back.
- **Schema control.** The schema is changed only by reviewed, versioned migrations; a test fails if the models and the migrations disagree.

Protection of files and generated documents

- Uploaded requirement files are stored privately in MinIO and are served only through the authorised API.
- Generated documents live in the project's GitHub repository, which has its own history (`git log`) — nothing can be changed without a trace. Repositories are public by an explicit, stated design decision of this educational project, and the upload screen warns users not to put confidential material in requirement documents.

Protection in transit and at the edge

- In production, all traffic uses **HTTPS**; the application is published through a Cloudflare tunnel, so the server opens no inbound port and the API binds only to the local interface.
- The frontend is served as a production build (not the development server) in production.
- **Production pre-flight.** The server refuses to start in production with a weak JWT secret, debug mode on, a missing public URL, a missing or invalid encryption key, or any seeded account still using its published demo password — and the refusal never prints the settings values. Demo accounts and demo data are not seeded in production. A command is provided to change a seeded password.

Protection of external accounts

- TRACE uses one **fine-grained GitHub token** scoped to the company Organization with only the permissions it needs (Administration, Contents, Issues read/write; Metadata read), not a classic full-access token, so a leaked token exposes only TRACE's repositories.
- The token cannot delete repositories; deleting a project **archives** its repository instead.
- Collaborators are invited with *Write* permission per repository, and access is revoked when a Project Manager leaves a project.
- The AI key, model and endpoint are fixed in `.env` and cannot be changed by any user through the application.

6.2 Creation of User Profiles and Access Rights

Creation of user profiles

- The first **Admin** account is created by the installation seed (in local development) or by the operator, and its password must be changed before production will start.
- **Admin and Manager accounts** can only be created by an Admin — through the Workforce screen or the registration endpoint called with an Admin token.
- **Public self-registration** always creates an **Employee** account; any requested privileged role is refused.
- Each user maintains their own profile — name, contact details, password and **GitHub username** — through the Settings page; a user can never edit another user's profile.
- The Admin maintains the workforce record (employee code, department, designation, shift, reporting manager, status) for each employee, individually or by CSV bulk upload. Setting a user *Inactive* or *Blocked* stops them from being chosen for projects.

Authentication

- Login returns a short-lived **access token** and a longer-lived **refresh token**, both JWTs signed with the server secret and carrying their type, so one cannot be used as the other.
- The browser sends the access token with every request; when it expires, the client uses the refresh token once to obtain a new one, and signs the user out if that fails.
- The WebSocket connection is authenticated with the same token; an invalid token is refused.

Access rights

Access is controlled in **two layers** on the server; the interface only reflects the result.

1. **Platform role on every route.** Every API route declares which of Admin, Manager and Employee may call it. An automated audit fails the build if any route has no guard; the only public routes are the health check and login, token refresh and registration.
2. **Project role on every project and task operation.** The caller's role in the specific project is resolved from `project_assignments`: the owning Manager administers the project; a Project Manager runs its structure; a Member works on their own tasks; any other Employee gets 403. Managers see only the projects they own; the Admin can read every project but change none of them through the project screens.

Table 49: Summary of access rights by role

Role	Rights
Admin	All administration: Control Panel, Workforce, Monitor, Configuration. Reads every project; takes no part in project delivery
Manager	Creates projects and uploads their documents; evaluates them; appoints and removes Project Managers; reads progress reports; discusses tasks of own projects; downloads reports. No HR functions
Project Manager	Edits the living project document; creates task groups and tasks; assigns Members; sets any task status (including completing on someone's behalf and cancelling); requests re-plans and suggestions
Member	Sees the project, tree and Gantt; works on their own tasks; sets their own task's status, including Completed; discusses their tasks
System	Sets *Overdue* (nightly sweep) — no person can

Status authority is enforced in one function on the server, and API responses carry the permissions (`can_edit`, `can_discuss`, `allowed_statuses`) derived from the same check, so a user can never be shown an action the server would refuse, and a crafted request for such an action is refused anyway.

A user who opens another role's panel sees a **403 page** that names their actual role, rather than being silently redirected, so a wrongly provisioned account is noticed immediately.

7. REPORTS

TRACE produces the reports below. Sample layouts follow.

Table 50: Reports generated by the system

Report	Trigger	Recipient / where
Project report (CSV) for a date range	On demand — *Download report* on the dashboard	Manager (own projects), Employee (projects they are on)
Project progress document	Weekly (Mondays 07:00) and on demand	Manager and team — Progress Report tab; repository file
Project report document	Weekly (Mondays 07:00) and on demand	Manager — Progress Report tab; repository file
Project document (initial and living)	Evaluate Project; edits by the PM	Everyone on the project — Overview; repository files
Task progress document	On task creation	Assignee and team — task page; repository file
Gantt chart version	Evaluation and every re-plan	Everyone on the project — Gantt tab, with version history
Dashboard statistics	Live	Manager and Employee dashboards
Notifications and e-mails	Assignment, due, overdue, completion, evaluation, discussion	The people concerned
Service health and job outcomes	Live	Admin — Monitor page

7.1 Project Report (CSV Download)

The dashboard's **Download report** action asks for a start date and an end date (and, optionally, one project) and downloads a CSV file named `trace-report-<start>-to-<end>.csv`. A task is included if it was created, due or completed within the range; each row shows its status as of today. The file opens in any spreadsheet program.

Table 51: Columns of the CSV project report

Column	Meaning	Example
Project	Project name	Storefront
Project status	Draft, Active, Completed, Archived	Active
Task group	Module the task belongs to	Backend Foundation
Group completion %	Completed ÷ non-cancelled tasks of the group	66.67
Task	Task title	Cart with coupons
Status	Status of the task today	In Progress

Column	Meaning	Example
Priority	Low, Medium, High, Critical	High
Assignee	Name of the Member	Dan K.
Created	Date the task was created	2026-09-05
Due	Due date	2026-09-20
Completed	Completion date, if completed	(empty)
Completed by	Assignee or Project Manager	(empty)
GitHub issue	Link to the task's issue	https://github.com/…/issues/3

Sample of the downloaded file:

```
Project,Project status,Task group,Group completion
%,Task,Status,Priority,Assignee,Created,Due,Completed,Completed by,GitHub issue
Storefront,Active,Backend Foundation,66.67,Auth API,Completed,High,Asha R.,2026-09-
01,2026-09-10,2026-09-09,Assignee,https://github.com/…/issues/1
Storefront,Active,Backend Foundation,66.67,Product catalogue API,Completed,Medium,Dan
K.,2026-09-02,2026-09-14,2026-09-15,Project Manager,https://github.com/…/issues/2
Storefront,Active,Backend Foundation,66.67,Cart with coupons,In Progress,High,Dan
K.,2026-09-05,2026-09-20,,,https://github.com/…/issues/3
Storefront,Active,Checkout,0.00,Payment gateway,Pending,Critical,,2026-09-12,2026-09-
30,,,https://github.com/…/issues/4
```

(Sample data for illustration.)

7.2 Project Progress Document and Report Document

Both are Markdown files written by the AI every week into the project's repository (`docs/project_progress_doc.md` and `docs/project_report_doc.md`) and rendered on the *Progress Report* tab with a **Rewrite now** button. Sample layout of the report document:

```
# Storefront — Weekly Report (week of 21 Sep 2026)

## Summary
Overall completion 42%. Backend Foundation is nearly done; Checkout has not started.
The project is on track for its planned end date of 30 Nov 2026.

## Progress by task group
| Task group          | Completion | Status      | Planned end |
|---------------------|-----------:|-------------|-------------|
| Backend Foundation  |   66.7 %   | In Progress | 25 Sep 2026 |
| Checkout            |    0.0 %   | Pending     | 20 Oct 2026 |

## Completed this week
- Auth API (Asha R.)
- Product catalogue API (Dan K., one day late)

## Risks and blockers
- Cart with coupons depends on the pricing rules still under discussion.
```

```
## Next week
- Finish the cart; start the payment gateway integration.
```

7.3 Task Progress Document

Written by the AI when a task is created (`docs/tasks/task-<id>.md`) and shown on the task page with its checklist:

```
# Task 3 — Cart with coupons

## Steps
- [ ] 1. Cart model — items, quantities, totals per user
- [ ] 2. Coupon rules — percentage and fixed discounts, expiry
- [ ] 3. API endpoints — add, remove, apply coupon, view cart
- [ ] 4. Tests — totals, invalid and expired coupons

## Acceptance criteria
- A user can add and remove items and see the correct total.
- An expired or unknown coupon is refused with a clear message.
```

7.4 Gantt Chart

The Gantt tab shows one bar per task group with its completion, dependency arrows, a *today* line and the project's end milestone; a bar expands to show its tasks as markers on their due dates. A version picker shows every earlier version with its number, date and reason (*initial*, *task_status_change*, *project_doc_updated*, *manual*).

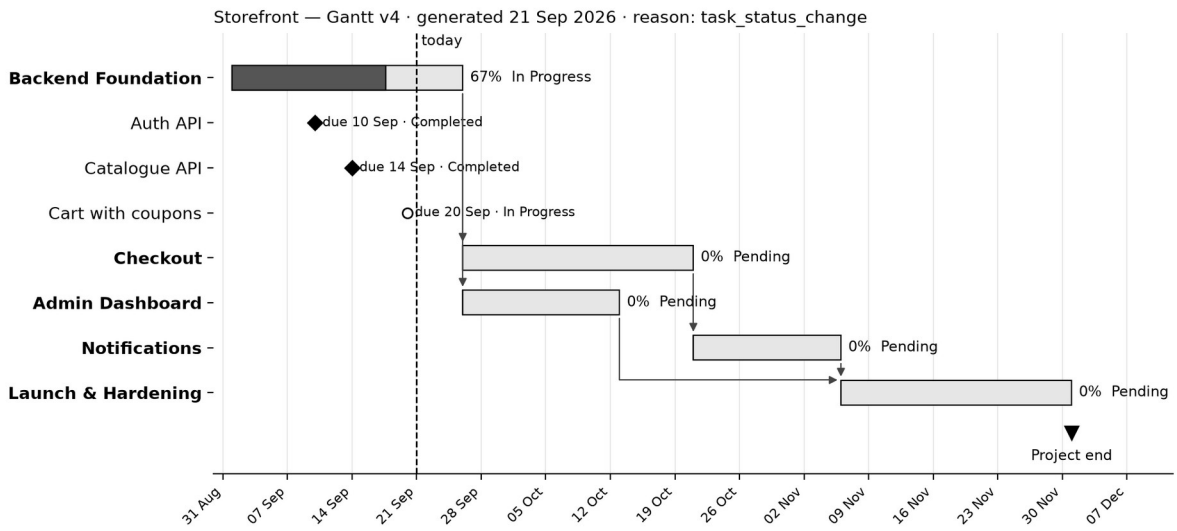


Figure 19: Sample layout — Gantt chart of a project as shown in TRACE

7.5 Dashboard Statistics

Table 52: Manager dashboard — statistics shown

Item	Content
Stat row	Total, active and completed projects; overall completion; number of at-risk projects
Project cards	Name, status, progress bar, Project Managers, open and overdue task counts
At-risk projects	Projects with overdue tasks or past their end date
Recent activity	Latest task changes across the Manager's projects

Table 53: Employee dashboard — statistics shown

Item	Content
Progress	Completion across the projects the Employee is on
My tasks	Open tasks grouped by due state: overdue, due soon, later, no date
Overdue callout	Count and list of overdue tasks
Notifications	Most recent notifications

7.6 Notification E-mail

```
Subject: New task: Cart with coupons

You were assigned 'Cart with coupons' in Storefront.

Open in TRACE: https://<trace-host>/employee/tasks/3
```

8. FUTURE SCOPE AND FURTHER ENHANCEMENT OF THE PROJECT

8.1 Limitations of the Present System

- **Public repositories only.** Every repository TRACE creates is public, so generated documents (including the AI's summary of the requirements) and issues are world-readable. This suits an educational deployment but not confidential client work.
- **One GitHub Organization per installation**, accessed with one token; personal GitHub accounts are not supported.
- **Dependence on external services.** Task creation needs GitHub, and the AI features need an AI provider; when either is unavailable those actions are refused (the rest of the system keeps working).
- **AI quality depends on the input.** The plan is only as good as the requirement documents; scanned PDFs without a text layer cannot be read (no OCR).
- **The task checklist is written once.** A task's progress document is not ticked off automatically as work proceeds; the task's status is the record.
- **No branch strategy.** Documents are committed to the default branch; per-task branches are not created.
- **Desktop-first interface.** The screens are designed for desktop browsers; there is no mobile application.
- **Members are not removed from projects**, and there is no *resend invitation* action; assigning a task again re-sends the invitation.
- **Two-factor authentication and session management** are not yet available.

8.2 Future Enhancements

- **Private repositories and per-project visibility**, with GitHub branch protection.
- **GitHub App integration** instead of a personal token — short-lived, per-installation credentials not tied to one person, and finer permissions.
- **Branch per task.** Create a branch named from the issue number when a task starts, and link pull requests to tasks.
- **Critical-path overlay on the Gantt chart**, with projected completion dates computed from dependencies.
- **Cross-project workload view** to detect people over-allocated across several projects at once, and use it in assignee ranking.

- **Scope-drift detection.** Compare the living project document and the work actually done against the immutable initial document, and warn when the project drifts from its original brief.
- **Chat-operations in task discussions** — for example `@trace create task …` to create a task in natural language.
- **Reusable project templates** — a standard task-group structure instantiated in one click.
- **Contribution heat-map** of commits and completions per person over time.
- **Multimodal intake** — whiteboard photos and voice notes as source documents, with OCR for scanned PDFs.
- **AI fairness audit** — periodic checks that suggestion scores show no systematic bias.
- **Explainable dispute queue** — if the AI is ever allowed to decide things, a way to contest its verdicts with a human override.
- **Mobile application** (React Native) and a responsive pass over all panels.
- **Two-factor authentication**, session management and single sign-on.
- **Integrations** with Slack or Microsoft Teams for notifications, and with calendars for due dates.
- **Export formats** — PDF and Excel versions of the project report, alongside CSV.
- **Multi-tenant SaaS deployment**, each company with isolated data and its own GitHub Organization.

9. BIBLIOGRAPHY

Books

- Pressman, Roger S. and Maxim, Bruce R. — *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
- Sommerville, Ian — *Software Engineering*, Pearson Education.
- Mall, Rajib — *Fundamentals of Software Engineering*, PHI Learning.
- Elmasri, Ramez and Navathe, Shamkant B. — *Fundamentals of Database Systems*, Pearson Education.
- Silberschatz, Abraham, Korth, Henry F. and Sudarshan, S. — *Database System Concepts*, McGraw-Hill Education.
- Lutz, Mark — *Learning Python*, O'Reilly Media.
- Banks, Alex and Porcello, Eve — *Learning React*, O'Reilly Media.

Standards

- IEEE Std 830-1998 — *IEEE Recommended Practice for Software Requirements Specifications*.
- RFC 7519 — *JSON Web Token (JWT)*, Internet Engineering Task Force.
- RFC 6455 — *The WebSocket Protocol*, Internet Engineering Task Force.

Official documentation

- FastAPI — https://fastapi.tiangolo.com
- Pydantic — https://docs.pydantic.dev
- SQLAlchemy 2.0 — https://docs.sqlalchemy.org
- Alembic — https://alembic.sqlalchemy.org
- PostgreSQL — https://www.postgresql.org/docs/
- Celery — https://docs.celeryq.dev
- Redis — https://redis.io/docs/
- React — https://react.dev
- Vite — https://vite.dev
- Tailwind CSS — https://tailwindcss.com/docs
- shadcn/ui — https://ui.shadcn.com
- React Flow (@xyflow/react) — https://reactflow.dev
- GitHub REST API — https://docs.github.com/en/rest
- OpenAI API reference — https://platform.openai.com/docs
- MinIO — https://min.io/docs
- Docker and Docker Compose — https://docs.docker.com

- Cryptography (Fernet) — https://cryptography.io
- pytest — https://docs.pytest.org

Other resources

- IGNOU MCA project guidelines (MCSP-232).
- Stack Overflow — https://stackoverflow.com
- MDN Web Docs — https://developer.mozilla.org

APPENDICES

Appendix A — Input and Output Screens

Screenshots of the running application are inserted here: Landing page, Login, 403 page; Admin — Control Panel, Workforce, Monitor, Configuration; Manager — Dashboard, Create Project wizard, Project Overview with the evaluation card, Tree, Gantt, Progress Report, Team with AI suggestions; Employee — Dashboard, Project Manager task board and Create Task dialog with suggestions, Project Document editor, Member's My Tasks, Task detail with progress document and discussion; the notification bell.

Appendix B — API Endpoint Reference

All endpoints are under `/api/v1`. The *Guard* column is the platform role checked on the route; project routes also check the project role.

Table 54: Principal API endpoints

Method	Path	Purpose	Guard
POST	/auth/login	Sign in; returns tokens and roles	Public
POST	/auth/refresh	New access token from a refresh token	Public
POST	/auth/register	Register (Employee only, unless called by an Admin)	Public
GET	/projects	Projects visible to the caller, with statistics	Any role
POST	/projects	Create a project and its repository	Manager
GET	/projects/report	CSV report for a date range	Manager, Employee
GET / PATCH / DELETE	/projects/{id}	Detail, edit, delete (archives repository)	Any / owner
POST / DELETE	/projects/{id}/source-files[/{fid}]	Upload with text extraction; remove	Owner
GET / POST / DELETE	/projects/{id}/assignments[/{uid}]	Team; appoint or remove a Project Manager	Owner
GET / POST / PATCH / DELETE	/projects/{id}/task-groups[/{gid}]	Task groups	Project Manager
GET / POST	/projects/{id}/tasks	List; create task and GitHub issue	Project Manager to create
GET	/tasks/mine	My open tasks across projects	Employee
GET / PATCH / DELETE	/tasks/{id}	Detail with permissions; edit details; delete	PM to change
PATCH	/tasks/{id}/assign	Assign / unassign	Project Manager
PATCH	/tasks/{id}/status	Change status (status authority)	Assignee or PM

Method	Path	Purpose	Guard
GET / POST	/tasks/{id}/discussions	Discussion thread	Participants
GET	/projects/{id}/tree	Project tree graph	Project users
GET	/projects/{id}/gantt[?version=N], /gantt/versions	Gantt and its history	Project users
POST	/projects/{id}/gantt/reevaluate	Re-plan now	Project Manager
POST / GET	/projects/{id}/evaluate[/status]	Evaluate Project; its state	Owner / project users
POST	/projects/{id}/suggest-managers	AI-ranked PM candidates	Owner
POST	/projects/{id}/tasks/suggest-assignees	AI-ranked assignees	Project Manager
POST	/projects/{id}/task-groups/{gid}/suggest-tasks	AI task suggestions	Project Manager
POST	/projects/{id}/progress-report	Rewrite progress and report documents now	Owner or PM
GET / PUT	/projects/{id}/documents/{kind}	Read a project document; save the living one	Project users / PM
GET	/tasks/{id}/progress-doc	The task's plan and checklist	Project users
GET / PATCH / POST	/notifications, /{id}/read, /read-all	Notification centre	Any role
WS	/ws?token=…	Live "refetch" events	Valid token
GET / PUT / POST	/admin/settings/github, /mail, …/test	Configuration and connection tests	Admin
—	/admin/tables, /workforce/{entity}[/bulk]	Control Panel CRUD; Workforce CRUD and bulk upload	Admin
GET	/services/health/*, /services/jobs/recent	Monitor	Admin
GET	/services/task/{job_id}/result	Poll a background job	Any role

Appendix C — Installation and Running

Prerequisites: Docker with Docker Compose, Python 3.11+, Node.js 20+, a GitHub Organization with a fine-grained token, and an OpenAI-compatible API key.

1. Copy `.env.example` to `.env` at the repository root and fill in the values (database, JWT secret, encryption key, AI key and model, GitHub organization and token, SMTP).
2. Install the backend dependencies into `backend/.venv` (`pip install -r requirements.txt`) and the frontend dependencies (`npm install` in `frontend/`).

3. Start everything with `./start.sh start` — this starts PostgreSQL, Redis, MinIO, Mailpit and pgAdmin in Docker, runs the migrations and seeds, and starts the API, the Celery worker and beat, and the frontend.
4. Open http://localhost:5173 and sign in as the Admin; set the GitHub organization and token in *Configuration* and press *Test connection*.

Table 55: Commands

Command	Purpose
`./start.sh start` / `stop` / `restart`	Start, stop or restart the whole stack
`./start.sh migrate`	Apply database migrations
`./start.sh status`	Show what is running
`./start.sh set-password <email>`	Set a user's password
`.venv/bin/python -m pytest -q` (in `backend/`)	Run the automated test suite

Table 56: Service addresses (local)

Service	Address
Frontend	http://localhost:5173
API and interactive API documentation	http://localhost:8000, http://localhost:8000/docs
pgAdmin	http://localhost:5050
Mailpit (development mail)	http://localhost:8025
MinIO console	http://localhost:9001

Table 57: Main environment settings (.env)

Key	Meaning
`APP_ENV`	`local` or `production`
`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `DB_PORT`	Database
`REDIS_PORT`	Redis (Celery broker, live events)
`MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET`	Object storage
`JWT_SECRET_KEY`	Token signing secret
`SETTINGS_ENCRYPTION_KEY`	Fernet key for stored secrets

Key	Meaning
`OPENAI_API_KEY`, `OPENAI_MODEL`, `OPENAI_BASE_URL`	AI model (fixed here; not changeable in the application)
`GITHUB_OWNER`, `GITHUB_TOKEN`, `GITHUB_COMMITTER_NAME`, `GITHUB_COMMITTER_EMAIL`	GitHub fallback values (Configuration overrides them)
`SMTP_FROM` and SMTP settings	Outgoing mail

Appendix D — Scheduled Jobs

Table 58: Celery beat schedule

Job	When	Purpose
`mark_overdue_tasks`	Every night, 23:00	Open past-due tasks → Overdue; notify; re-plan affected Gantt charts
`task_due_reminders`	Every day, 08:00	Remind assignees and PMs of tasks due tomorrow
`update_project_progress_docs`	Mondays, 07:00	AI rewrites every active project's progress and report documents
`compute_all_weekly_scores`	Sundays, 23:30	Task-based performance scores used as evidence in AI ranking

GLOSSARY

Table 59: Glossary of terms

Term	Meaning
Access token	Short-lived signed token sent with every request to prove who the user is
Alembic	Tool that applies versioned changes (migrations) to the database schema
API	Application Programming Interface — the set of HTTP endpoints the frontend calls
ASGI	Asynchronous Server Gateway Interface — the standard FastAPI runs on
Assignee	The Member a task is assigned to; the person doing it
bcrypt	Slow, salted password-hashing algorithm
Celery	Distributed task queue that runs background jobs
Celery beat	Scheduler that queues jobs at fixed times
Critical path	The longest chain of dependent activities; it fixes the project's duration
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values — a plain-text table format
Debounce	Collapsing a burst of triggers into a single action
DFD	Data Flow Diagram
Docker Compose	Tool that runs several containers (database, Redis, MinIO …) together
ER diagram	Entity Relationship diagram
Evaluate Project	The AI action that turns source documents into a project document, task groups and a Gantt chart
FastAPI	Python web framework used for the backend
Fernet	Authenticated symmetric encryption used for stored secrets
Fine-grained token	A GitHub token limited to chosen repositories and permissions
Gantt chart	Bar chart of activities on a calendar
GitHub issue	A work item in a GitHub repository; in TRACE, one per task
GitHub Organization	Shared GitHub account owning the repositories TRACE creates
JSON	JavaScript Object Notation — the data format of the API
JSON mode	AI model option that forces the reply to be a JSON object
JSONB	PostgreSQL binary JSON column type
JWT	JSON Web Token (RFC 7519)
LLM	Large Language Model — the AI model used for planning and writing
Markdown	Plain-text formatting language used for the generated documents
Member	Employee assigned at least one task in a project
Migration	A versioned, reversible change to the database schema

Term	Meaning
MinIO	S3-compatible object store for uploaded files
ORM	Object-Relational Mapping — working with tables as classes (SQLAlchemy)
PERT	Programme Evaluation and Review Technique
Project document	The AI-written description of the project; an immutable initial copy and a living copy
Project Manager	Employee appointed to run one project
Pydantic	Python library that validates data against declared schemas
Re-plan	Regeneration of the Gantt chart as a new version
Redis	In-memory data store used as the job broker and for live events
Refresh token	Longer-lived token used only to obtain a new access token
REST	Representational State Transfer — style of HTTP API
Slug	URL-safe form of a name, used as the repository name
Soft delete	Marking a row deleted (`deleted_at`) instead of removing it
SPA	Single-Page Application
SRS	Software Requirement Specification
Status authority	The rule deciding who may set a task's status
Task group	A module of work in a project; one Gantt bar
Vite	Build tool and development server for the frontend
WebSocket	Persistent two-way connection used for live events